

ARCHON AGENTIC OS 2026 — LISTA COMPLETA DE RECURSOS

Catálogo de sistema & módulos, visão geral da arquitetura e matriz de funções

Atualizado em: 11 de novembro de 2026 | Versão: v0.5.1 ("Archon Hardened") | Idioma: Português

Catálogo de sistema & módulos, visão geral da arquitetura e matriz de funções

© Michael Keller

Sumário

1. Versão & visão geral de releases (v0.5.1) 3
2. Visões do sistema & estúdios especiais (UI Views) 3
3. Ecossistema de economia de tokens & FinOps (minimização de custos) 8
4. Kernel, agendamento & arquitetura de daemons (Pilar 1) 11
5. Modelo de processo multiagente, hierarquia & síntese (Pilar 2) 12
6. Drivers MCP, serviços externos & ferramentas multimodais (Pilar 3) 14
7. Memória, higiene de banco de dados & Obsidian 2nd Brain (Pilar 4) 15
8. Segurança, governança zero-trust & diagnóstico do sistema (Pilar 6) 17
9. Autoevolução, marketplaces & autodocumentação (Pilar 7) 19
10. Matriz de referência: plugins Evey/Hermes controláveis 19
11. Arquitetura enterprise & de resiliência 21
12. Arquitetura de autonomia, sandbox & governança 23
13. Núcleo de performance, escala de throughput & eficiência de tokens 24
14. Arquitetura moderna de memória & RAG 26
15. Execução confiável & determinismo (durable execution) 27
16. Segurança, privacidade & guardrails enterprise 28
17. Auto-otimização, avaliação & inteligência de enxame 29
18. Pilar 3: Economia & ERP (alfa) & suíte de mandantes 31
19. Recursos de arquitetura, segurança & deployment (v0.5.0 – v0.5.1) 31
20. Glossário alfabético & índice remissivo 33

1. Versão & visão geral de releases (v0.5.1)

Versão	Codinome	Principais marcos de arquitetura & recursos
v0.5.1	Archon Hardened	Routers FastAPI modulares (main.py -88%), 25 achados críticos + 68 altos de segurança corrigidos, proxy Docker-admin classe A, gestão de usuários RBAC (PBKDF2), atualização do sistema em 2 fases (?local_only=true), integração de proxy AnythingLLM, instalador portátil em 1 clique.
v0.5.0	Economia & ERP (alfa)	Pilar 3 completo (sincronização CRM/PocketBase, itens/serviços, processos ANG/AB, contabilidade com DATEV & GiroCode, mandantes corporate), Research Lab v2 & 55 frameworks de boardroom.
v0.3.x	Sentinel / Foundation	Kernel de SO de 7 pilares, corpo de 70 especialistas, 34+ plugins Evey, roteamento FinOps em 3 níveis, sandboxes efêmeras de código, depuração por replay, PII-Scrubber, cache semântico & busca híbrida FTS5.

2. Visões do sistema & estúdios especiais (UI Views)

1. Visão: Mission Control (cockpit de monitoramento & centro de comando)

Problema: Sem um cockpit central em tempo real, faltam visão de agentes ativos, vazão de tokens, custos diários e liberações bloqueadas.

Mecânica: Agrupa 5 cartões vitais de KPI (carga da frota, vazão TPM, vigia de custos, estabilidade, portão de aprovação), a projeção do quadro em 5 colunas, a barra de inspeção de processos ativos (rodapé de 3 slots para heartbeat, concorrência de ferramentas e saturação de contexto), a faixa de atividade de tokens de 15 minutos (sparkline SVG) e o stream de auditoria ao vivo.

Benefício: *Fornece monitoramento 360° sem atraso de todo o sistema operacional em uma única tela.*

Preço & lado sombrio (trade-off): Como visão central permanente / infraestrutura do sistema integrada ao dashboard (necessidade básica de RAM/CPU desprezível; custos de tokens surgem apenas na execução ativa de ações de LLM dentro do módulo).

2. Visão: Operações – quadro Kanban & engine de tarefas

Problema: Tarefas em históricos puros de chat se perdem e não podem ser gerenciadas nem por status nem por responsabilidades.

Mecânica: 5 colunas de ciclo de vida (a fazer, em processamento, revisão & QA, bloqueada, concluída) direto em kanban.db (fonte única da verdade). Suporta limites WIP configuráveis por coluna, filtros de prioridade (P0–P3), atribuição de agentes, registro de horas e vinculação Obsidian em 1 clique. Integra o portão de qualidade de 4 olhos: antes de mover uma tarefa para 'done', um revisor par (Katharina Völk para QM/conformidade ou Robert) verifica o resultado quanto a integralidade, substância e capacidade de ação (placar de 1–100%). Com 'changes_requested', uma trava de status atua; um override human-in-the-loop (force=True) prevalece se necessário.

Benefício: *Estrutura todos os projetos operacionais com transparência e evita sobrecarga do sistema por barreiras de trabalho em andamento.* Evita trabalhos inacabados no status 'done' por revisão obrigatória de 4 olhos.

Preço & lado sombrio (trade-off): Como visão central permanente / infraestrutura do sistema integrada ao dashboard (necessidade básica de RAM/CPU desprezível; custos de tokens surgem apenas na execução ativa de ações de LLM dentro do módulo).

3. Visão: Operações – organograma 2D interativo & linhas de reporte

Problema: Listas planas de agentes não mostram poderes de instrução nem rotas de escalonamento em processos bloqueados.

Mecânica: Visualiza 56 nós e 55 linhas de reporte como grafo 2D interativo (apex -> CEO Robert -> líderes técnicos -> especialistas). Suporta drag & drop para trocar superiores, alinhamento à grade, visão de cartões de equipe e salvamento permanente em config/org_hierarchy.json.

© Michael Keller

Benefício: *Define cadeias vinculantes de escalonamento e delegação que o sistema segue automaticamente em bloqueios.*

Preço & lado sombrio (trade-off): Como visão central permanente / infraestrutura do sistema integrada ao dashboard (necessidade básica de RAM/CPU desprezível; custos de tokens surgem apenas na execução ativa de ações de LLM dentro do módulo).

4. Visão: Operações – comunicação interna entre agentes (tráfego de rádio)

Problema: Quando especialistas passam subtarefas entre si, isso ou inunda o chat do usuário ou permanece invisível.

Mecânica: Central dedicada de escuta com lista de canais (#allgemein, #tech-dev, #legal, #marketing) e feed cronológico de mensagens de todos os acordos entre agentes de kanban.db.

Benefício: *Torna a colaboração da frota de agentes rastreável sem sobrecarregar a caixa postal primária do usuário.*

Preço & lado sombrio (trade-off): Como visão central permanente / infraestrutura do sistema integrada ao dashboard (necessidade básica de RAM/CPU desprezível; custos de tokens surgem apenas na execução ativa de ações de LLM dentro do módulo).

5. Visão: entrada & triagem (gatekeeper HITL de 3 colunas & caixa postal)

Problema: Ações descontroladas de escrita de agentes (p. ex. envio de e-mails) e entradas dispersas de e-mail e Telegram trazem riscos de segurança.

Mecânica: Central de triagem em três colunas: coluna 1 para liberações human-in-the-loop marcadas em amarelo (liberar/rejeitar), coluna 2 para sincronização de e-mail IMAP/SMTP com triagem automática & delegação, coluna 3 para o stream ao vivo espelhado do Telegram (@Beatman_bot).

Benefício: *Garante que nenhuma operação externa de escrita seja executada sem liberação expressa do operador.*

Preço & lado sombrio (trade-off): Como visão central permanente / infraestrutura do sistema integrada ao dashboard (necessidade básica de RAM/CPU desprezível; custos de tokens surgem apenas na execução ativa de ações de LLM dentro do módulo).

6. Visão: corpo de agentes & gestão de frotas (cockpit de 8 abas)
Problema: especialistas precisam de modelos, prompts de sistema, ferramentas e instruções de trabalho individuais em um local central.
Mecânica: gerencia 70 perfis de especialistas com filtros rápidos (ativos, favoritos, todos, 7 domínios), hot-swap de provedor & modelo e 8 abas técnicas: visão geral (dossiê 360° & apelido no Telegram), chat direto, habilidades (skills), operações (histórico de tarefas), SOUL (editor de prompt do sistema), dossiê (SOPs), mercado de talentos e mercado de skills.
Benefício: departamento central de pessoal para calibragem e ampliação em segundos de todo o quadro de agentes.
Preço & lado sombrio (trade-off): como visão central permanente / infraestrutura do sistema integrada ao dashboard (necessidade básica de RAM/CPU desprezível; custos de tokens surgem apenas na execução ativa de ações de LLM dentro do módulo).

7. Visão: Master Chat (workspace de modo duplo & barra lateral multissessão)

Problema: Ao trabalhar diretamente com agentes, faltava um ambiente focado de tela cheia com alternância rápida entre diálogo 1:1, chat em grupo e missão.

Mecânica: Oferece o interruptor de modo duplo ('direto' para resposta imediata vs. 'missão' para criação estruturada de tarefas), upload de múltiplos arquivos (até 5 arquivos), captura de tela do navegador (botão de câmera), entrada de voz (ditado), seletor de orçamento de raciocínio (0 a 16k tokens), otimizador de prompts de IA e histórico de sessões à direita (agentes vs. grupo). Inclui o pipeline de voz do Telegram: mensagens de voz recebidas (.ogg/Opus) em trânsito são transcritas via Whisper/Gemini, roteadas por apelidos (/robert, /anwalt etc.) ao agente técnico e respostas via edge-tts (de-DE-ConradNeural) são reenviadas direto ao Telegram como memorandos naturais de voz.

Benefício: *Une diálogo de alta velocidade, entrada multimodal e controle de missão em uma superfície de trabalho contínua.* Permite operação completa por voz móvel e troca bidirecional de áudio via Telegram.

Preço & lado sombrio (trade-off): Como visão central permanente / infraestrutura do sistema integrada ao dashboard (necessidade básica de RAM/CPU desprezível; custos de tokens surgem apenas na execução ativa de ações de LLM dentro do módulo).

8. Visão: AI Mastermind & Boardroom (55 frameworks de metodologia)

Problema: decisões estratégicas complexas exigem métodos estruturados de gestão e perspectivas controversas em vez de opiniões isoladas.

Mecânica: simula sessões de colegiado em múltiplas etapas (chairman + 3 slots reais de agentes técnicos + desafiante red-team) com base em 34 frameworks selecionáveis em 7 categorias (35 base) (p. ex. CLEAR, Design Sprint, Double Diamond, Lean Startup, First Principles, Game Theory, Blue Ocean, pré-mortem, Six Sigma).

Benefício: impõe rigor metodológico, revela pontos cegos pelo red-team e entrega memorandos de board prontos para decisão.

Preço & lado sombrio (trade-off): como visão central permanente / infraestrutura do sistema integrada ao dashboard (necessidade básica de RAM/CPU desprezível; custos de tokens surgem apenas na execução ativa de ações de LLM dentro do módulo).

9. Visão: Showroom (explorador de projetos, galeria de artefatos & sandbox Python)

Problema: Relatórios gerados, protótipos HTML, cálculos e scripts se perdem em conversas longas.

Mecânica: Cataloga todos os resultados de trabalho por criador, data e tipo. Inclui o leitor de cofre do workspace com renderização ao vivo para Markdown/HTML/JSON e uma sandbox Python isolada para execução segura de código. Expandido com o executor ao vivo interativo do Showroom: permite a execução direta de arquivos Python e de código no navegador com timeout de 12s, diretório scratch isolado e gaveta de terminal expansível (STDOUT, STDERR, código de saída e tempo em milissegundos).

Benefício: *Serve como salão de aceite e apresentação de todos os deliverables prontos antes do uso pelo cliente ou em produção.* Artefatos de código e scripts podem ser verificados e testados direto no Showroom sem trocar de console.

Preço & lado sombrio (trade-off): Como visão central permanente / infraestrutura do sistema integrada ao dashboard (necessidade básica de RAM/CPU desprezível; custos de tokens surgem apenas na execução ativa de ações de LLM dentro do módulo).

10. Visão: biblioteca central & 2nd Brain (RAG, auto-OCR & Obsidian Vault)

Problema: livros técnicos, normas DIN e planos de urbanização em PDF não podiam até agora ser pesquisados por agentes sem cópia manual.

Mecânica: une 3 subabas e 4 hubs centrais (00_HR_Agenten, 02_Wissensbibliothek, Protokolle/, 00_Notebook_Synthesen): 1. índice RAG AnythingLLM (porta 3001), 2. capturador de documentos com drag & drop, auto-OCR (Docling) e classificação temática por IA, 3. árvore direta de arquivos Obsidian & visualizador Markdown incl. alternador de cofre.

Benefício: transforma pareceres em PDF de forma plenamente automática em conhecimento Markdown e vetorial pesquisável para os 70 agentes.

Preço & lado sombrio (trade-off): como visão central permanente / infraestrutura do sistema integrada ao dashboard (necessidade básica de RAM/CPU desprezível; custos de tokens surgem apenas na execução ativa de ações de LLM dentro do módulo).

11. Visão: engine de workflow (n8n Automation Mesh na porta 5678)

Problema: Processos recorrentes de interface (cron-jobs, webhooks, sincronizações de banco de dados) consomem tokens desnecessariamente quando rodam via LLMs.

Mecânica: Incorpora o editor visual de workflows n8n diretamente e o conecta via skill n8n-automation-mesh (FastMCP) ao agente CEO Robert. Rotinas de 0 token ativadas: 1. briefing matinal (dias úteis 07:30 com arquivamento Obsidian e entrega via Telegram), 2. manutenção semanal (domingos 23:00 com backup SQLite

& auditoria de saúde), 3. watchdog de RSS & notícias (varredura horária), mais 5 webhooks funcionais ativos para extração web, mensagens, renderização de documentos, busca de feeds e sincronização de CRM.

Benefício: *Combina processos determinísticos de 0 token em segundo plano com decisões cognitivas de agentes em pontos-chave.*

Preço & lado sombrio (trade-off): Como visão central permanente / infraestrutura do sistema integrada ao dashboard (necessidade básica de RAM/CPU desprezível; custos de tokens surgem apenas na execução ativa de ações de LLM dentro do módulo).

12. Visão: gateway LLM (dashboard do roteador OmniRoute AI)

Problema: A vinculação fixa a um único provedor de nuvem leva a parada imediata do sistema em caso de limites de taxa ou falhas.

Mecânica: Dashboard central de proxy (porta 20128 / 8000) com cofre de múltiplos provedores, medição de latência em tempo real (ms), monitor de erros HTTP (429/500), disjuntor automático e cascata de fallback em 4 etapas (GLM-4.7 Flash -> Gemini 2.5 Flash -> Claude 3.5 Sonnet -> Llama 3.3 70B local).

Benefício: *Garante 99,9% de disponibilidade de todos os processos cognitivos com custos mínimos de API.*

Preço & lado sombrio (trade-off): Como visão central permanente / infraestrutura do sistema integrada ao dashboard (necessidade básica de RAM/CPU desprezível; custos de tokens surgem apenas na execução ativa de ações de LLM dentro do módulo).

13. Visão: Token Guard (guardião & análise de tokens em 3 subvisões)

Problema: Falta de transparência sobre qual agente, modelo ou sessão individual causa mais tokens e custos.

Mecânica: Oferece filtros de tempo (24h, 7d, 30d, total), temporizador de inatividade e 3 subvisões: 1. visão geral & gráfico de tendência diária com tabelas de perfil/modelo e formulário de barreiras de proteção, 2. inspetor de consumo de sessão individual (tokens de entrada, saída, leitura de cache ordenáveis), 3. recursos & limitação de tarefas (controle de 1–16 tarefas).

Benefício: *Controle total de custos até a session-ID individual incl. limitação imediata de orçamento e paralelismo.*

Preço & lado sombrio (trade-off): Como visão central permanente / infraestrutura do sistema integrada ao dashboard (necessidade básica de RAM/CPU desprezível; custos de tokens surgem apenas na execução ativa de ações de LLM dentro do módulo).

14. Visão: extensões & central de módulos (34 plugins especiais Evey)

Problema: Funções de segurança, memória e análise não podiam até agora ser ligadas ou desligadas modularmente por área técnica.

Mecânica: Controla 34 plugins especializados Hermes/Evey em 8 abas técnicas: segurança & guardiões (4), performance & cache (2), monitoramento & análises (3), inteligência de agentes (6), planejamento & tarefas (2), pontes & interfaces (4), memória & conhecimento (3), ferramentas & execução (3) mais finanças/qualidade/manutenção. Inclui o sistema padronizado de plugins & hooks de runtime: estrutura modular de pastas (plugins/<plugin_id>/plugin.json), 4 hooks de ciclo de vida (on_load, on_enable, on_disable, on_unload) e 2 interceptadores de ferramentas (before_tool_execute, after_tool_execute) com plugins iniciais pré-instalados (métricas do sistema, gancho de auditoria de segurança, guarda de custos).

Benefício: *Permite ligar e desligar com precisão capacidades individuais do sistema em operação por clique.* Permite extensões isoladas de runtime e interceptação global de ferramentas sem intervir no código do kernel.

Preço & lado sombrio (trade-off): Como visão central permanente / infraestrutura do sistema integrada ao dashboard (necessidade básica de RAM/CPU desprezível; custos de tokens surgem apenas na execução ativa de ações de LLM dentro do módulo).

15. Visão: Research Lab (busca profunda & triangulação de evidências)

Problema: Consultas simples entregam ocorrências isoladas não verificadas sem cruzamento de fontes ou avaliação de confiança.

Mecânica: Cockpit de pesquisa em múltiplas etapas com profundidade selecionável, monitor de fontes, placar

de evidências, protocolo de síntese ao vivo para triangulação de declarações contraditórias e exportação em 1 clique ao Obsidian Vault. Inclui o loop autônomo de pesquisa profunda & reflexão v2 (loop de repesquisa): decomposição de pesquisas complexas em 3–5 subperguntas, análise autônoma de lacunas, até 3 buscas subsequentes iterativas, fallback com múltiplos provedores (SearXNG / Jina Reader) e proteção SSRF.

Benefício: *Gera dossiês de mercado, jurídicos e técnicos sólidos e citáveis.* Fecha lacunas de conhecimento por conta própria por pesquisas subsequentes recursivas em caso de fontes contraditórias ou incompletas.

Preço & lado sombrio (trade-off): Como visão central permanente / infraestrutura do sistema integrada ao dashboard (necessidade básica de RAM/CPU desprezível; custos de tokens surgem apenas na execução ativa de ações de LLM dentro do módulo).

16. Visão: telemetria & logs (stream de auditoria ao vivo de 3 fontes)

Problema: Em caso de erros em contêineres ou prompts de agentes, era preciso até agora buscar laboriosamente no terminal via comandos Docker.

Mecânica: Agrupa 3 fontes de logs em tempo real: logs físicos de contêineres Docker (stdout/stderr dos 11 contêineres), logs cognitivos de agentes e a trilha de auditoria criptográfica. Com filtro de nível (DEBUG, INFO, WARN, ERROR), trava de rolagem automática e exportação.

Benefício: *Caixa-preta incorruptível para diagnóstico de falhas em segundos e comprovações de conformidade.*

Preço & lado sombrio (trade-off): Como visão central permanente / infraestrutura do sistema integrada ao dashboard (necessidade básica de RAM/CPU desprezível; custos de tokens surgem apenas na execução ativa de ações de LLM dentro do módulo).

17. Estúdio especial: Archon Prompt Studio (832 modelos & SOP Fabric)

Problema: tarefas formuladas com imprecisão levam a alucinações, formatos errados de saída e loops caros de correção.

Mecânica: kit modular integrado com 824 modelos enterprise em 16 categorias técnicas, mais de 255 padrões Fabric-SOP e a fórmula de 5 campos (1. função, 2. contexto, 3. tarefa, 4. formato, 5. restrições/regras negativas), além de otimizador de prompts de IA.

Benefício: força cada delegação a uma estrutura cristalina e a transfere com um clique ao chat ou tarefa Kanban.

Preço & lado sombrio (trade-off): como visão central permanente / infraestrutura do sistema integrada ao dashboard (necessidade básica de RAM/CPU desprezível; custos de tokens surgem apenas na execução ativa de ações de LLM dentro do módulo).

18. Estúdio especial: Visual CSS Live Studio & gizmo de transformação in-place (Alt + S)

Problema: Ajustes de design exigiam até agora buscar em arquivos CSS e recarregar o navegador constantemente.

Mecânica: Estúdio flutuante de design em tempo real (Alt + S) com inspetor DOM de mira, gizmo interativo de transformação (arrastar posição, escalar 4 cantos/bordas, arrastar arredondamento), gerenciador de perfis de tema e exportação direta de CSS para custom_studio.css.

Benefício: *Permite estilização ao vivo precisa até o pixel de toda a interface em operação sem recarregar a página.*

Preço & lado sombrio (trade-off): Como visão central permanente / infraestrutura do sistema integrada ao dashboard (necessidade básica de RAM/CPU desprezível; custos de tokens surgem apenas na execução ativa de ações de LLM dentro do módulo).

19. Estúdio especial: Design System Lab, arquiteto AppShell & estúdio de 15 componentes

Problema: Alturas de cabeçalho, espaçamentos ou contrastes divergentes entre modo claro e escuro perturbam a calma visual.

Mecânica: Inclui o arquiteto interativo AppShell (alinhamento de cabeçalho de 56px, largura da sidebar, padding de viewport com simulação de miniatura ao vivo) e o estúdio de design de 15 componentes (os-workspace a os-badge) com engine de tema duplo (claro/escuro).

Benefício: *Garante uma aparência B2B-SaaS uniforme e sem ofuscamento em todas as 20 visões.*

Preço & lado sombrio (trade-off): Como visão central permanente / infraestrutura do sistema integrada ao dashboard (necessidade básica de RAM/CPU desprezível; custos de tokens surgem apenas na execução ativa de ações de LLM dentro do módulo).

20. Visões especiais: System Control Center (11 abas técnicas, dock ao vivo) & omnibox global (Ctrl + K)
Problema: as configurações estavam dispersas; a navegação entre 20 visões e 70 agentes exigia muitos cliques.
Mecânica: o System Control Center agrupa 7 abas de administração (Hermes Core incl. editor YAML, contêineres & serviços, provedor & gateway, metodologias, plugins, comunicação, design) e pode ser ancorado à direita via 'Live-Dock'. A omnibox (Ctrl + K) navega pelo teclado de imediato a cada aba (#), agente (@) ou comando. Integra o sistema de atualização em 1 clique & deploy contínuo: sincronização automática com GitHub com snapshots pré-update de todos os bancos SQLite (WAL) em data/snapshots/ antes de cada pull do Git, rollback em 1 clique e verificações mínimas de git-stat contra timeouts de montagem.
Benefício: velocidade máxima de operação para administradores e power-users. Permite atualizações seguras de software com rollback automático de banco de dados em caso de erro.
Preço & lado sombrio (trade-off): como visão central permanente / infraestrutura do sistema integrada ao dashboard (necessidade básica de RAM/CPU desprezível; custos de tokens surgem apenas na execução ativa de ações de LLM dentro do módulo).

3. Ecossistema de economia de tokens & FinOps (minimização de custos)

21. Limites automáticos de roteamento de custos em 3 níveis (LOW / MEDIUM / HIGH)

Problema: Quando tarefas simples de rotina rodam em modelos caros de ponta (como Claude Sonnet ou GPT-4o), os custos diários explodem.

Mecânica: Limites configuráveis de preço por 1 milhão de tokens no System Control Center: LOW (máx. $0,10/1M -> impõe free-tier/flash-lite), MEDIUM (máx. $1,00/1M -> Haiku/Gemini Flash), HIGH (sem limite -> melhores modelos de raciocínio para arquitetura complexa).

Benefício: *Reduz os gastos correntes com API no dia a dia em até 85% sem perder qualidade em tarefas difíceis.*

Preço & lado sombrio (trade-off): Como visão central permanente / infraestrutura do sistema integrada ao dashboard (necessidade básica de RAM/CPU desprezível; custos de tokens surgem apenas na execução ativa de ações de LLM dentro do módulo).

22. Proxy Docker da API FreeLLM & roteamento 'auto/best-free' (porta 3002)

Problema: Cotas gratuitas de diferentes provedores são difíceis de monitorar manualmente e mudam com frequência.

Mecânica: Um contêiner Docker dedicado (freellmapi na porta 3002) agrupa endpoints LLM gratuitos. O perfil de roteamento 'auto/best-free' seleciona automaticamente o modelo mais forte disponível de custo zero.

Benefício: *Permite a operação contínua de agentes de segundo plano e triagem a exatamente $0,00 de custos de API.*

Preço & lado sombrio (trade-off): Como visão central permanente / infraestrutura do sistema integrada ao dashboard (necessidade básica de RAM/CPU desprezível; custos de tokens surgem apenas na execução ativa de ações de LLM dentro do módulo).

23. Controle contínuo de orçamento de raciocínio no Master Chat & roteador (0 a 16k tokens)

Problema: Modelos de raciocínio (modo thinking) frequentemente consomem milhares de tokens invisíveis de raciocínio para perguntas triviais.

Mecânica: Controle em 6 níveis (desligado: 0 tok, mínimo: ~1k, baixo: ~2k, médio: ~4k, alto: ~8k, máximo: ~16k) no rodapé do chat e em agent_router.py.

Benefício: *Desliga completamente loops caros de raciocínio em tarefas simples e reserva raciocínio profundo para casos realmente complexos.*

Preço & lado sombrio (trade-off): Como visão central permanente / infraestrutura do sistema integrada ao

dashboard (necessidade básica de RAM/CPU desprezível; custos de tokens surgem apenas na execução ativa de ações de LLM dentro do módulo).

24. Evey-Cache & controle de TTL de cache (cache de respostas semântico & exato de 24h)

Problema: Consultas recorrentes idênticas ou muito similares ao sistema levavam a cada vez a novas chamadas de API pagas.

Mecânica: O plugin evey-cache armazena respostas de modelos com duração configurável (padrão: TTL de 24 horas) e mostra separadamente no inspetor de sessões os tokens economizados de leitura de cache.

Benefício: *Responde consultas repetidas em 0 milissegundos a 0 custo de tokens.*

Preço & lado sombrio (trade-off): Se o TTL do cache estiver alto demais (> 72h), o sistema entrega respostas desatualizadas em temas dinâmicos.

Quando desligar? & limites: Desligar em consultas em tempo real (cotações ao vivo, status do servidor). TTL padrão: 24h.

25. Limitador rígido de Max-Context & Max-History no guardião (32k / 20 turns)

Problema: Sessões de chat em crescimento infinito reenviam a cada nova pergunta todo o histórico e elevam os tokens de entrada exponencialmente.

Mecânica: Barreiras rígidas ajustáveis no guardião para a janela máxima de contexto (p. ex. 32.000 tokens) e o número máximo de mensagens de chat transmitidas (p. ex. últimos 20 turns).

Benefício: *Limita de forma confiável os custos máximos possíveis de entrada por turno.*

Preço & lado sombrio (trade-off): Corta mensagens acima de 32k tokens ou 20 turns de forma rígida, eliminando contexto muito antigo.

Quando desligar? & limites: Para análises profundas de documentos, elevar para 64k–128k no Hermes Core.

26. Recortador dinâmico de tokens para modelos de baixo contexto (backend/token_clipper.py)

Problema: Modelos compactos ou locais (8k/16k) falham quando grandes saídas de ferramentas estouram a janela de contexto.

Mecânica: Preserva obrigatoriamente o prompt do sistema (índice 0) e a última instrução do usuário, encurta blocos individuais longos demais com inteligência e remove etapas intermediárias mais antigas.

Benefício: *Permite o uso estável de micromodelos extremamente baratos mesmo em tarefas extensas.*

Preço & lado sombrio (trade-off): Corta saídas de ferramentas longas demais no meio ([...SNIP...]), de modo que linhas individuais de código podem faltar no texto bruto.

Quando desligar? & limites: Desativar brevemente quando logs brutos completos forem necessários sem cortes.

27. Compressão de memória por janela deslizante (contexto consolidado de decisão)

Problema: Ao simplesmente cortar mensagens antigas, decisões anteriores do projeto se perdem.

Mecânica: Condensa rodadas antigas de conversa em um resumo compacto e o injeta como bloco [CONTEXTO CONSOLIDADO DE CONVERSA & DECISÃO] na posição 1.

Benefício: *Economiza até 70% de tokens históricos de conversa preservando todas as decisões tomadas.*

Preço & lado sombrio (trade-off): A etapa de compressão custa uma única vez ~800 tokens e suaviza formulações literais de rodadas anteriores.

Quando desligar? & limites: Se necessário, ajuste fino via protect_last_n (padrão: 20) no Hermes Core.

28. Troca de páginas LRU-K para RAM de subagentes (backend/context_swapper.py)

Problema: Subagentes paralelos mantêm enormes quantidades de dados brutos simultaneamente na memória ativa de prompts.

Mecânica: Externaliza páginas de contexto não mais urgentemente necessárias a partir de 80% de carga pelo princípio least-recently-used para SQLite e as recarrega apenas sob demanda (reidratação).

Benefício: *Mantém os prompts enxutos e evita taxas desnecessárias de tokens por conhecimento de fundo inativo.*

Preço & lado sombrio (trade-off): O swap e recarregamento de RAM de agentes para disco gera ~10–25 ms de latência de I/O na troca de contexto.

Quando desligar? & limites: Manter ativo a partir de 80% de uso de RAM para evitar travamentos OOM.

29. Roteador JEV — pré-filtragem determinística de intenção de 0 token (jev_router.py)

Problema: Classificar cada instrução simples do sistema por um grande modelo de linguagem desperdiça tokens e tempo.

Mecânica: Classifica entradas recebidas antes da chamada LLM por vetor de execução JSON determinístico e heurística de palavras-chave.

Benefício: *Encaminha comandos claros do sistema direto à rotina-alvo sem consumir um único token de LLM.*

Preço & lado sombrio (trade-off): Como visão central permanente / infraestrutura do sistema integrada ao dashboard (necessidade básica de RAM/CPU desprezível; custos de tokens surgem apenas na execução ativa de ações de LLM dentro do módulo).

30. Prompter ASD-STE100 de inglês técnico simplificado (.agents/skills/asd-ste100-prompter)

Problema: Textos prolixos em prosa em prompts do sistema e instruções de agentes incham desnecessariamente cada chamada de API.

Mecânica: Transforma instruções segundo o padrão aeronáutico ASD-STE100 (um significado por palavra, verbos ativos, frases curtas, zero enchimento).

Benefício: *Reduz o comprimento em tokens de prompts do sistema em aprox. 30–40% e ao mesmo tempo diminui a taxa de erro em chamadas de ferramentas.*

Preço & lado sombrio (trade-off): Como visão central permanente / infraestrutura do sistema integrada ao dashboard (necessidade básica de RAM/CPU desprezível; custos de tokens surgem apenas na execução ativa de ações de LLM dentro do módulo).

31. Anti-AI-Slop & compressão de saída (.agents/skills/anti-ai-slop)

Problema: Modelos de linguagem geram por padrão longos floreios de cortesia, introduções e resumos redundantes (tokens caros de saída).

Mecânica: Suprime clichês de IA, frases vazias e inchaço artificial direto no prompt do sistema e limpa saídas.

Benefício: *Economiza até 25% dos tokens de saída especialmente caros e entrega textos técnicos imediatamente utilizáveis.*

Preço & lado sombrio (trade-off): Como visão central permanente / infraestrutura do sistema integrada ao dashboard (necessidade básica de RAM/CPU desprezível; custos de tokens surgem apenas na execução ativa de ações de LLM dentro do módulo).

32. Estimador de custos pré-voo & previsão de orçamento (backend/cost_estimator.py)

Problema: Antes de iniciar grandes tarefas, não era previsível quanto a execução custaria.

Mecânica: Calcula antecipadamente via POST /api/router/estimate-cost os tokens esperados de entrada/saída e o valor em dólar com base na tabela atual de preços de modelos e o concilia com o orçamento da tarefa.

Benefício: *Alerta sobre execuções caras antes mesmo do primeiro centavo gasto.*

Preço & lado sombrio (trade-off): Como visão central permanente / infraestrutura do sistema integrada ao dashboard (necessidade básica de RAM/CPU desprezível; custos de tokens surgem apenas na execução ativa de ações de LLM dentro do módulo).

33. Orçamento dinâmico de tarefas com downgrade econômico de 80% & parada de 100%

Problema: Uma única tarefa complexa podia consumir despercebidamente todo o orçamento diário.

Mecânica: Monitora os custos acumulados por tarefa (p. ex. limite $0,25): reduz automaticamente a um modelo econômico barato aos 80% e pausa a tarefa aos 100% com solicitação de liberação via Telegram.

Benefício: *Evita estouros de custo em pesquisas autônomas de múltiplas etapas.*

Preço & lado sombrio (trade-off): Ao atingir 80% do orçamento de tarefas, reduz-se para um modelo econômico mais barato; aos 100% ocorre a parada rígida.

Quando desligar? & limites: Elevar o orçamento de tarefas (padrão: $0,50) para $2,00 em grandes execuções de arquitetura.

34. Eliminador de loops & limite máx. de etapas (15 etapas / Jaccard > 90%)

Problema: Quando um agente entra em loop infinito em uma chamada defeituosa de ferramenta, ele queima crédito a cada segundo.

Mecânica: Verifica cada etapa quanto a repetições exatas (>= 3) ou referências circulares semânticas (> 90% de similaridade) e interrompe tarefas no mais tardar no limite configurável de etapas máximas (padrão: 15 etapas).

Benefício: *Protege a conta de forma confiável contra loops infinitos despercebidos durante a noite.*

Preço & lado sombrio (trade-off): Limites rígidos demais (máx. idênticos = 2 ou Jaccard < 0,80) interrompem cedo demais tarefas iterativas legítimas como correção gradual de testes ou paginação.

Quando desligar? & limites: Desligar brevemente quando um agente deve executar deliberadamente muitas etapas uniformes de loop (p. ex. migração em massa de 50 arquivos). | Limites: máx. idênticos: 2 = para retries | > 5 = loops caros (padrão: 3). Jaccard: < 0,80 = falsos alarmes | > 0,96 = detecta loops tardiamente (padrão: 0,90). Máx. de etapas: padrão 15.

4. Kernel, agendamento & arquitetura de daemons (Pilar 1)

35. Hermes Master Orchestrator (núcleo Hermes)
Problema: agentes individuais descoordenados trabalham uns contra os outros ou perdem o objetivo geral de vista.
Mecânica: central de controle de processos para análise de pedidos, seleção de funções, monitoramento e aceite de resultados.
Benefício: garante coerência estratégica entre os 70 especialistas.
Preço & lado sombrio (trade-off): mais serviços ativos em segundo plano e raciocínio alto ('high') aumentam a carga básica e os custos de tokens.
Quando desligar? & limites: controlável granularmente pelas 14 seções na aba 'HERMES CORE'.

36. Daemon de kernel tickless & loop de eventos asyncio (portas 8420 / 8501)

Problema: Polling constante a cada segundo desperdiça recursos de CPU e energia do sistema.

Mecânica: Calcula o momento exato da próxima tarefa devida ('sleep-until-next-due') e coloca o kernel em repouso até lá.

Benefício: *Carga mínima do sistema em repouso com reação de despertar sem atraso.*

Preço & lado sombrio (trade-off): Como visão central permanente / infraestrutura do sistema integrada ao dashboard (necessidade básica de RAM/CPU desprezível; custos de tokens surgem apenas na execução ativa de ações de LLM dentro do módulo).

37. Agendador de preempção por prioridade (fila P0–P3)

Problema: Perguntas urgentes de usuários no chat direto precisavam esperar quando jobs em segundo plano ocupavam todos os slots.

Mecânica: Fila priorizada: tarefas P3 (chat direto / emergência) ultrapassam imediatamente jobs de rotina P1/P2 em andamento.

Benefício: *Garante tempos de resposta rápidos no diálogo apesar da alta carga em segundo plano.*

Preço & lado sombrio (trade-off): Como visão central permanente / infraestrutura do sistema integrada ao dashboard (necessidade básica de RAM/CPU desprezível; custos de tokens surgem apenas na execução ativa de ações de LLM dentro do módulo).

38. Limitação de concorrência livremente escalável (backend/concurrency_throttle.py)

Problema: Agentes simultâneos demais levam a limites de taxa HTTP-429 e picos de 99% de CPU.

Mecânica: Regula workers ativos por um semáforo asyncio (ajustável por controle de 1 a 16 tarefas paralelas no guardião e no Control Center).

Benefício: *Permite operação protegida com limitação em laptops assim como operação de alta carga em workstations.*

Preço & lado sombrio (trade-off): Limites restritos de estagnação (< 30 s) interrompem erroneamente modelos lentos de deep-reasoning (p. ex. o3/R1) no meio do raciocínio; mais workers aumentam picos de RAM e API.

Quando desligar? & limites: Desligar em análises individuais extremamente longas ou execuções locais de LLM em hardware lento. | Limites: máx. de workers: 1–2 = acúmulo | > 8 = picos de RAM & HTTP-429 (padrão: 4). Estagnação: < 30s = interrompe pensadores | > 180s = correção tardia (padrão: 45s).

39. Watchdog autônomo de 30 segundos & recuperação de falhas (watchdog_daemon.py)

Problema: Tarefas travadas ou congeladas ficavam permanentemente presas no status in_progress.

Mecânica: Verifica a cada 30 segundos todos os processos em execução, cancela tarefas zumbis após timeout (120s) e transfere tarefas órfãs à recuperação automática de falhas.

Benefício: *Mantém todos os slots de workers livres de registros mortos.*

Preço & lado sombrio (trade-off): Como visão central permanente / infraestrutura do sistema integrada ao dashboard (necessidade básica de RAM/CPU desprezível; custos de tokens surgem apenas na execução ativa de ações de LLM dentro do módulo).

40. Supervisor de deadlocks & estagnação (backend/deadlock_supervisor.py)

Problema: Dependências mútuas entre dois agentes podem levar a um congelamento silencioso.

Mecânica: Detecta estados circulares de espera e tarefas sem progresso (> 45s) e resolve o bloqueio de forma autônoma com backoff exponencial.

Benefício: *Evita paradas de pipeline em trabalhos complexos em grupo.*

Preço & lado sombrio (trade-off): Um limite de estagnação restrito demais (< 30s) interrompe modelos lentos de deep-reasoning (o3/R1) no meio do raciocínio.

Quando desligar? & limites: Ao usar modelos de raciocínio, elevar o limite de estagnação para 90–120s (padrão: 45s).

41. Autocorreção de frotas & reinício automático de contêineres (backend/fleet_self_healing.py)

Problema: Se um dos 11 contêineres Docker falhasse, precisava ser reiniciado manualmente.

Mecânica: Monitora todos os contêineres centrais via docker-socket-proxy protegido e realiza em caso de falhas (> 60s) uma reinicialização autônoma incl. entrada de auditoria.

Benefício: *Garante a operação 24/7 não supervisionada de todo o cenário Docker.*

Preço & lado sombrio (trade-off): Um contêiner permanentemente defeituoso pode entrar em loop de reinício por auto-restart.

Quando desligar? & limites: Desativar brevemente durante manutenções manuais de contêineres.

42. Standup matinal proativo & serviço de despertar (backend/autonomous_standup.py)

Problema: De manhã faltava uma visão compacta de execuções noturnas, custos e decisões pendentes.

Mecânica: Gera diariamente às 08:00 um relatório consolidado de status no Obsidian Vault (01_PROJEKTE/Daily_Briefings/) e o envia ao dashboard e ao Telegram.

Benefício: *Começa cada dia de trabalho com um modelo de decisão pronto.*

Preço & lado sombrio (trade-off): Cada briefing matinal automático consome ~1.500–3.000 tokens no horário configurado.

Quando desligar? & limites: Desligar quando não se deseja relatório matinal diário (padrão: 08:00).

5. Modelo de processo multiagente, hierarquia & síntese (Pilar 2)

43. 70 perfis declarativos de agentes & dossiês Markdown
Problema: assistentes genéricos de IA não têm consciência de função nem padrões definidos de qualidade.
Mecânica: cada um dos 70 especialistas está ancorado em agent_profiles.json e em um dossiê próprio (00_HR_Agenten/) com prompt do sistema, rota de modelo, escopo de ferramentas e SOPs.

Benefício: cria profissionais virtuais confiáveis para estratégia, direito, construção, finanças, marketing e TI.
Preço & lado sombrio (trade-off): como visão central permanente / infraestrutura do sistema integrada ao dashboard (necessidade básica de RAM/CPU desprezível; custos de tokens surgem apenas na execução ativa de ações de LLM dentro do módulo).

44. Delegação Robert, sincronização fan-in & contrato de marcadores Lisa (SYNTHESE-BEREIT)

Problema: Em subtarefas paralelas, resultados parciais se perdiam ou eram resumidos de forma incompleta.

Mecânica: Robert decompõe grandes projetos; um lock de processo (_MC_DATA_LOCK) aguarda a conclusão de todas as subtarefas (fan-in). Lisa agrega os relatórios verificando o marcador obrigatório SYNTHESE-BEREIT; Robert concede na linha 1 a LIBERAÇÃO formal.

Benefício: *Garantia de qualidade em múltiplas etapas nos moldes de uma gestão empresarial real.*

Preço & lado sombrio (trade-off): Como visão central permanente / infraestrutura do sistema integrada ao dashboard (necessidade básica de RAM/CPU desprezível; custos de tokens surgem apenas na execução ativa de ações de LLM dentro do módulo).

45. Mesa de handoff padronizada agente-para-agente (backend/handoff_desk.py)

Problema: Perdas silenciosas de contexto na passagem de um especialista ao próximo.

Mecânica: Gera protocolos formais de passagem com deliverables, suposições abertas e checksum SHA-256, que devem ser confirmados pelo agente receptor.

Benefício: *Interfaces à prova de auditoria entre todos os departamentos.*

Preço & lado sombrio (trade-off): Verificações rígidas de SLA rejeitam respostas factuais muito curtas, porém corretas, quando min_content_length está alto demais (> 300).

Quando desligar? & limites: Padrão: 100 caracteres de comprimento mínimo, 60 min de turnaround.

46. Agendador DAG em duas etapas para execução paralela de projetos (backend/dag_scheduler.py)

Problema: O processamento sequencial de muitas subtarefas demora demais em grandes projetos.

Mecânica: Calcula o grafo acíclico dirigido (DAG): executa pesquisas independentes (etapa 1) em paralelo e inicia depois automaticamente a síntese (etapa 2).

Benefício: *Reduz o tempo de execução de análises complexas em múltiplas vezes.*

Preço & lado sombrio (trade-off): Como visão central permanente / infraestrutura do sistema integrada ao dashboard (necessidade básica de RAM/CPU desprezível; custos de tokens surgem apenas na execução ativa de ações de LLM dentro do módulo).

47. Autocorreção & loop de reflexão de execução de ferramentas

Problema: Erros fugazes de sintaxe ou parâmetros em chamadas de ferramentas levavam ao cancelamento imediato.

Mecânica: Captura erros de ferramentas, reflete a mensagem de erro e corrige os parâmetros de chamada de forma autônoma em até 3 tentativas.

Benefício: *Robustez significativamente maior em operações de arquivos, bancos de dados e web.*

Preço & lado sombrio (trade-off): Como visão central permanente / infraestrutura do sistema integrada ao dashboard (necessidade básica de RAM/CPU desprezível; custos de tokens surgem apenas na execução ativa de ações de LLM dentro do módulo).

48. Workflows de longa duração interruptíveis com snapshots de estado SQLite

Problema: Se uma tarefa espera horas por uma liberação do usuário, ela ocupa memória desnecessariamente ou perde seu estado em uma reinicialização.

Mecânica: Congela o estado exato de execução na tabela workflow_checkpoints e retoma o trabalho sem interrupção após a liberação.

Benefício: *Permite processos de negócio de vários dias através de reinicializações.*

Preço & lado sombrio (trade-off): Como visão central permanente / infraestrutura do sistema integrada ao

dashboard (necessidade básica de RAM/CPU desprezível; custos de tokens surgem apenas na execução ativa de ações de LLM dentro do módulo).

49. Portão de consenso de dois agentes & revisão crítica adversarial (backend/consensus_gate.py)

Problema: Um único agente frequentemente não percebe seus próprios erros de raciocínio ou alucinações em tarefas críticas.

Mecânica: Impõe em tarefas P3 e intervenções críticas uma revisão formal por um agente revisor independente (red-team/crítico). No quadro Kanban conectado como portão automatizado de qualidade de 4 olhos (revisão Katharina Völk / Robert com placar 1–100% e trava de status em caso de necessidade de revisão).

Benefício: *Princípio dos 4 olhos para todos os resultados críticos ao negócio.*

Preço & lado sombrio (trade-off): Triplica (3x) o consumo de tokens e aumenta o tempo de resposta em +6 a +20 s; a unanimidade traz risco de deadlock.

Quando desligar? & limites: Ativar apenas a partir da prioridade P3/P4 e eleger maioria de 2/3.

6. Drivers MCP, serviços externos & ferramentas multimodais (Pilar 3)

50. Ecossistema Archon CRM & PocketBase (portas 3005, 8090, 8092)

Problema: Contatos de clientes, históricos de e-mail e assets de mídia ficavam separados dos agentes de IA.

Mecânica: Frontend dedicado de CRM (porta 3005) com backend PocketBase (porta 8090) e servidor de mídia (porta 8092), totalmente conectado via scripts/archon_crm_mcp.py.

Benefício: *Agentes podem buscar contatos, pesquisar a fundo, avaliar (lead scoring) e atribuir correspondências.*

Preço & lado sombrio (trade-off): Como visão central permanente / infraestrutura do sistema integrada ao dashboard (necessidade básica de RAM/CPU desprezível; custos de tokens surgem apenas na execução ativa de ações de LLM dentro do módulo).

51. Correspondência automática de e-mails & importador de arquivo do LinkedIn

Problema: Milhares de contatos de exportações do LinkedIn e caixas de e-mail precisavam ser conciliados manualmente.

Mecânica: Sincroniza caixas postais (--fetch-mails), atribui e-mails automaticamente a contatos CRM existentes e importa CSVs completos de conexões do LinkedIn incl. deduplicação.

Benefício: *Constrói em minutos um banco de dados relacional completo e enriquecido.*

Preço & lado sombrio (trade-off): Como visão central permanente / infraestrutura do sistema integrada ao dashboard (necessidade básica de RAM/CPU desprezível; custos de tokens surgem apenas na execução ativa de ações de LLM dentro do módulo).

52. Gerente de permissões de ferramentas Robert (backend/tool_permission_manager.py)

Problema: Agentes não devem cancelar impotentes ante pacotes Python/Node ausentes, mas também não podem instalar nada sem serem solicitados.

Mecânica: Robert apresenta um pedido formal de instalação (pending_permission) com comando e justificativa; só após sua liberação em 1 clique o pacote é instalado.

Benefício: *Extensibilidade controlada com proteção total do sistema host.*

Preço & lado sombrio (trade-off): Como visão central permanente / infraestrutura do sistema integrada ao dashboard (necessidade básica de RAM/CPU desprezível; custos de tokens surgem apenas na execução ativa de ações de LLM dentro do módulo).

53. Malha de webhooks orientada a eventos (backend/event_mesh.py)

Problema: Eventos externos de GitHub, formulários ou sistemas de terceiros não podiam disparar ações de agentes.

Mecânica: Receptor universal de webhooks que valida payloads JSON externos e os converte direto em tarefas Kanban executáveis.

Benefício: *Conecta o Archon OS em tempo real ao restante do cenário de TI.*

Preço & lado sombrio (trade-off): Como visão central permanente / infraestrutura do sistema integrada ao dashboard (necessidade básica de RAM/CPU desprezível; custos de tokens surgem apenas na execução ativa de ações de LLM dentro do módulo).

54. Hub central de notificações multicanal (backend/notification_hub.py)

Problema: Avisos importantes se perdiam na UI, enquanto mensagens irrelevantes incomodavam no celular.

Mecânica: Roteia alertas estritamente por gravidade: INFO -> toast de UI, WARNING -> log do dashboard, CRITICAL -> push imediato do Telegram.

Benefício: *Alarmes precisos sem fadiga de notificações.*

Preço & lado sombrio (trade-off): Como visão central permanente / infraestrutura do sistema integrada ao dashboard (necessidade básica de RAM/CPU desprezível; custos de tokens surgem apenas na execução ativa de ações de LLM dentro do módulo).

55. Pipeline multimodal de visão & captura de tela (backend/vision_pipeline.py)

Problema: Problemas visuais ou tabelas em imagens precisavam ser descritos laboriosamente.

Mecânica: Captura direta de tela no navegador (botão de câmera no chat) e análise automática de OCR e estrutura para imagens carregadas (.png, .jpg, .webp).

Benefício: *Os agentes veem exatamente o que você vê na tela.*

Preço & lado sombrio (trade-off): Como visão central permanente / infraestrutura do sistema integrada ao dashboard (necessidade básica de RAM/CPU desprezível; custos de tokens surgem apenas na execução ativa de ações de LLM dentro do módulo).

56. Apollo Voice Engine — TTS de borda & ditado Whisper (apollo_voice_engine.py)

Problema: Digitar textos longos no smartphone ou desktop custa tempo; relatórios quer-se ouvir em trânsito.

Mecânica: Pipeline bidirecional de voz: ditado por microfone (speech-to-text) direto no campo de entrada do chat e saída natural de voz (text-to-speech) para briefings. Integração total de voz do Telegram (POST /api/telegram/voice-relay e POST /api/voice/synthesize) para memorandos móveis de áudio via edge-tts (de-DE-ConradNeural).

Benefício: *Transforma o Archon OS em um assistente executivo controlado por voz.*

Preço & lado sombrio (trade-off): Como visão central permanente / infraestrutura do sistema integrada ao dashboard (necessidade básica de RAM/CPU desprezível; custos de tokens surgem apenas na execução ativa de ações de LLM dentro do módulo).

57. Oracle News Radar & agente de execução de SEO

Problema: Mudanças de mercado e fraquezas de SEO dos próprios sites eram verificadas apenas esporadicamente.

Mecânica: O Oracle News Radar varre continuamente fontes técnicas quanto à relevância para projetos; o agente de SEO verifica páginas quanto a meta-tags, estrutura e densidade de palavras-chave.

Benefício: *Otimização contínua de mercado e visibilidade em segundo plano.*

Preço & lado sombrio (trade-off): Como visão central permanente / infraestrutura do sistema integrada ao dashboard (necessidade básica de RAM/CPU desprezível; custos de tokens surgem apenas na execução ativa de ações de LLM dentro do módulo).

7. Memória, higiene de banco de dados & Obsidian 2nd Brain (Pilar 4)

58. Camada central de memória SQLite WAL (5 tabelas padrão)

Problema: Acessos simultâneos de leitura e escrita de vários agentes levavam a conflitos em arquivos simples.

Mecânica: Banco SQLite transacional (data/agentic_memory.db) em modo write-ahead-logging (WAL) com as 5 tabelas centrais tasks, conversations, learnings, kv_store e user_profile.

Benefício: *Acessos paralelos sem bloqueio com máxima segurança de dados.*

Preço & lado sombrio (trade-off): Como visão central permanente / infraestrutura do sistema integrada ao dashboard (necessidade básica de RAM/CPU desprezível; custos de tokens surgem apenas na execução ativa de ações de LLM dentro do módulo).

59. Sidecar Kanban à prova de processos com troca atômica de arquivos (backend/kanban_sidecar.py)

Problema: Metadados como limites WIP ou tags podiam ser corrompidos em operações de escrita paralelas.

Mecânica: Protege config/kanban_meta.json por um mutex de processo e operações atômicas de write-replace.

Benefício: *Evita perda de dados em atualizações frequentes do quadro.*

Preço & lado sombrio (trade-off): Como visão central permanente / infraestrutura do sistema integrada ao dashboard (necessidade básica de RAM/CPU desprezível; custos de tokens surgem apenas na execução ativa de ações de LLM dentro do módulo).

60. Ponte Docker-RAG para o AnythingLLM (5.068 documentos indexados na porta 3001)

Problema: O conhecimento existente no armazenamento vetorial Docker muitas vezes não era consultado automaticamente em perguntas normais de chat.

Mecânica: backend/rag_bridge.py conecta o núcleo do agente diretamente ao workspace ARCHON do AnythingLLM e entrega trechos relevantes como contexto fundamentado.

Benefício: *Respostas fundamentadas com base em seus documentos empresariais reais em vez de estimativas genéricas.*

Preço & lado sombrio (trade-off): Como visão central permanente / infraestrutura do sistema integrada ao dashboard (necessidade básica de RAM/CPU desprezível; custos de tokens surgem apenas na execução ativa de ações de LLM dentro do módulo).

61. Disposição de pastas de projeto em 3 níveis no Obsidian Vault

Problema: Quando agentes criam arquivos, a falta de regras fixas gera rapidamente desordem no cofre.

Mecânica: Cria automaticamente para cada projeto a estrutura em 3 níveis: 01_Zwischenberichte, 02_Medien e 03_Endprodukte.

Benefício: *Separação clara entre rascunhos e resultados finais aprovados.*

Preço & lado sombrio (trade-off): Como visão central permanente / infraestrutura do sistema integrada ao dashboard (necessidade básica de RAM/CPU desprezível; custos de tokens surgem apenas na execução ativa de ações de LLM dentro do módulo).

62. Indexador do Obsidian Vault & mapa automático de conteúdo (backend/vault_hygiene.py)

Problema: Novos relatórios de projeto frequentemente ficavam sem links no grafo Obsidian ('notas órfãs').

Mecânica: Varre o cofre, detecta notas órfãs e atualiza de forma plenamente automática o _MOC_Projekt_Index.md central com wikilinks.

Benefício: *Mantém o grafo de conhecimento no Obsidian sempre totalmente conectado.*

Preço & lado sombrio (trade-off): Como visão central permanente / infraestrutura do sistema integrada ao dashboard (necessidade básica de RAM/CPU desprezível; custos de tokens surgem apenas na execução ativa de ações de LLM dentro do módulo).

63. Coleta autônoma de fatos & deduplicação por 85% de similaridade (backend/post_task_pipeline.py)

Problema: Aprendizados importantes de tarefas concluídas se perdiam ou eram salvos em duplicata várias vezes.

Mecânica: Extrai automaticamente após a conclusão da tarefa novos fatos para learnings e user_profile e evita duplicatas a partir de 85% de similaridade textual.

Benefício: *O sistema fica mais inteligente a cada tarefa concluída, sem entulhar a memória.*

Preço & lado sombrio (trade-off): Como visão central permanente / infraestrutura do sistema integrada ao dashboard (necessidade básica de RAM/CPU desprezível; custos de tokens surgem apenas na execução ativa de ações de LLM dentro do módulo).

64. Backup online automático de BD com rotação de 7 dias (backend/db_backup_manager.py)

Problema: Sem backups regulares do banco de dados, defeitos de hardware ameaçam a perda de todas as tarefas e perfis.

Mecânica: Executa em operação PRAGMA wal_checkpoint(TRUNCATE) e salva arquivos compactados .db.gz em .hermes/backups/db/.

Benefício: *Backup integral sem um segundo de inatividade.*

Preço & lado sombrio (trade-off): Backups completos diários ocupam ~50–500 MB de disco conforme o tempo de retenção.

Quando desligar? & limites: Padrão: 7 dias de retenção (db_backup_keep_days).

65. Vacuum SQLite autônomo & otimizador de fragmentação (backend/db_optimizer.py)

Problema: Registros excluídos e updates frequentes incham arquivos SQLite desnecessariamente ao longo de meses.

Mecânica: Executa via POST /api/system/optimize-db em todos os bancos de dados PRAGMA optimize e VACUUM e calcula o espaço liberado.

Benefício: *Mantém os tempos de consulta permanentemente na faixa de milissegundos.*

Preço & lado sombrio (trade-off): Como visão central permanente / infraestrutura do sistema integrada ao dashboard (necessidade básica de RAM/CPU desprezível; custos de tokens surgem apenas na execução ativa de ações de LLM dentro do módulo).

66. Snapshot atômico pré-escrita & rollback em 1 clique (backend/snapshot_manager.py)

Problema: Quando um agente sobrescrevia um arquivo com erro, a restauração do estado anterior era trabalhosa.

Mecânica: Salva automaticamente o original em .hermes/snapshots/ antes de cada acesso de escrita e o restaura de imediato por clique ou API. Adicionalmente integrados como snapshots pré-update de todos os bancos SQLite antes de pulls do GitHub e atualizações de contêineres em data/snapshots/.

Benefício: *Rede de segurança total em todas as alterações de arquivos e código.*

Preço & lado sombrio (trade-off): Cada snapshot pré-escrita custa ~5–15 ms de I/O de disco antes de sobrescrever um arquivo.

Quando desligar? & limites: Buffer circular limitado por padrão a 50 snapshots.

67. Prévia de diff ao vivo compatível com Git (backend/diff_engine.py)

Problema: Antes de uma liberação, quer-se ver exatamente quais linhas mudam, em vez de reler o arquivo inteiro.

Mecânica: Gera uma comparação colorida de linhas (vermelho = removido, verde = novo) entre o acervo e o rascunho do agente.

Benefício: *Verificação em segundos de alterações de código e documentos no portão HITL.*

Preço & lado sombrio (trade-off): Como visão central permanente / infraestrutura do sistema integrada ao dashboard (necessidade básica de RAM/CPU desprezível; custos de tokens surgem apenas na execução ativa de ações de LLM dentro do módulo).

8. Segurança, governança zero-trust & diagnóstico do sistema (Pilar 6)

68. Log de auditoria SHA-256 encadeado criptograficamente (backend/audit_log.py)

Problema: Logs de texto simples podem ser alterados ou excluídos posteriormente.

Mecânica: Cada evento é encadeado ao hash SHA-256 da entrada anterior (princípio blockchain) e pode ser verificado quanto a manipulação via GET /api/audit/verify.

Benefício: *Comprovação à prova de auditoria de todas as ações de sistema e liberação.*

Preço & lado sombrio (trade-off): Overhead mínimo de escrita (~2 ms por entrada); a cadeia criptográfica cresce ao longo de meses.

Quando desligar? & limites: Manter permanentemente ligado para governança à prova de auditoria.

69. Guarda de segurança local — 25 assinaturas de prompt-injection & jailbreak

Problema: E-mails ou páginas maliciosas podem conter comandos ocultos ('Ignore previous instructions...').

Mecânica: Verifica cada entrada externa de texto contra 25 padrões de ataque antes que chegue ao contexto de um agente.

Benefício: *Bloqueia prompt-injections indiretas na fronteira do sistema.*

Preço & lado sombrio (trade-off): Heurísticas muito rígidas podem disparar falsos alarmes (falsos positivos) em textos de segurança de TI ou prompts de pen-testing.

Quando desligar? & limites: Relaxar brevemente apenas para testes autorizados de red-teaming.

70. Mascaramento de segredos & guarda de saída red-teaming (backend/secret_masker.py)

Problema: Chaves de API ou senhas podem aparecer acidentalmente em logs, relatórios ou saídas de chat.

Mecânica: Filtra textos de saída em tempo real por padrões de chave (sk-..., Bearer, senhas) e os substitui por [GESCHÜTZTER_KEY].

Benefício: *Evita a divulgação acidental de credenciais confidenciais.*

Preço & lado sombrio (trade-off): Mascaramento agressivo por regex pode, em casos raros, ocultar strings inofensivas de Base64 ou hash em saídas de código.

Quando desligar? & limites: Em operação produtiva, manter permanentemente ativo para proteger chaves de API.

71. Proxy protegido de socket Docker (porta 2375 interna)

Problema: Acesso direto a /var/run/docker.sock dá aos contêineres direitos irrestritos de root sobre o sistema host.

Mecânica: Interpõe um contêiner proxy restritivo que só deixa passar comandos explícitos permitidos de status e reinício.

Benefício: *Protege o sistema operacional host segundo o princípio de confiança zero.*

Preço & lado sombrio (trade-off): Como visão central permanente / infraestrutura do sistema integrada ao dashboard (necessidade básica de RAM/CPU desprezível; custos de tokens surgem apenas na execução ativa de ações de LLM dentro do módulo).

72. Comparador de modelos A/B & pontuador empírico de performance (backend/model_benchmarker.py)

Problema: Novos modelos de linguagem surgem semanalmente; sua adequação a tarefas empresariais próprias muitas vezes permanece incerta.

Mecânica: Testa dois modelos em paralelo com tarefas idênticas e compara tempo de resposta, custos de tokens e qualidade do resultado.

Benefício: *Base objetiva de dados para a escolha de modelos no corpo de agentes.*

Preço & lado sombrio (trade-off): Como visão central permanente / infraestrutura do sistema integrada ao dashboard (necessidade básica de RAM/CPU desprezível; custos de tokens surgem apenas na execução ativa de ações de LLM dentro do módulo).

73. Placar de saúde 360° em todo o sistema & engine de diagnóstico (backend/system_diagnostics.py)

Problema: Métricas isoladas muitas vezes não mostram de relance quão saudável o sistema está como um todo.

Mecânica: Calcula a partir do status da frota Docker (35%), fila de concorrência (20%), integridade de auditoria (20%), ausência de deadlocks (15%) e acessibilidade RAG (10%) um placar geral de 0–100% incl. recomendações de ação.

Benefício: *Diagnóstico geral imediato do sistema operacional via GET /api/system/health-score.*

Preço & lado sombrio (trade-off): Como visão central permanente / infraestrutura do sistema integrada ao dashboard (necessidade básica de RAM/CPU desprezível; custos de tokens surgem apenas na execução ativa de ações de LLM dentro do módulo).

9. Autoevolução, marketplaces & autodocumentação (Pilar 7)

74. Síntese autônoma de skills com feedback de dossiês (backend/skill_synthesizer.py)

Problema: Caminhos de solução bem-sucedidos em múltiplas etapas precisavam até agora ser programados manualmente como nova skill.

Mecânica: Reconhece cadeias recorrentes bem-sucedidas de ferramentas, gera automaticamente um rascunho de SKILL.md, o registra no dossiê do agente e o apresenta para liberação.

Benefício: *O sistema amplia suas capacidades em operação de forma autônoma.*

Preço & lado sombrio (trade-off): Como visão central permanente / infraestrutura do sistema integrada ao dashboard (necessidade básica de RAM/CPU desprezível; custos de tokens surgem apenas na execução ativa de ações de LLM dentro do módulo).

75. Mercado dinâmico de talentos & skills (skill_markt.js & AgentSkillsHub)

Problema: Escrever novas funções ou workflows para novas áreas de negócio do zero consome muito tempo.

Mecânica: Catálogo pesquisável para importação e exportação em 1 clique de perfis completos de agentes e skills externas (incl. integração agentskillshub.dev).

Benefício: *Escala a frota de agentes em novos projetos em segundos.*

Preço & lado sombrio (trade-off): Como visão central permanente / infraestrutura do sistema integrada ao dashboard (necessidade básica de RAM/CPU desprezível; custos de tokens surgem apenas na execução ativa de ações de LLM dentro do módulo).

76. Sincronização total frontend-backend (238 fetches / 581 rotas / 29 modais)

Problema: Em grandes aplicações web, botões órfãos ou modais desconectados frequentemente levam a erros silenciosos.

Mecânica: Pipeline automatizada de verificação garante que todas as 238 chamadas de API de frontend e 29 modais estejam exatamente conectados às 368 rotas de backend.

Benefício: *Garante 0 links mortos e 100% de controles operáveis em todas as visões.*

Preço & lado sombrio (trade-off): Como visão central permanente / infraestrutura do sistema integrada ao dashboard (necessidade básica de RAM/CPU desprezível; custos de tokens surgem apenas na execução ativa de ações de LLM dentro do módulo).

77. Gerador autônomo de dossiê mestre & manual (backend/system_dossier_generator.py)

Problema: Documentações de arquitetura geralmente ficam obsoletas poucos dias após sua criação.

Mecânica: Gera ao toque de botão (POST /api/system/generate-dossier) o dossiê atual de arquitetura do sistema simultaneamente na pasta do projeto e no Obsidian Vault (00_DOKUMENTATION/).

Benefício: *Mantém a documentação do sistema permanentemente atualizada sem esforço manual de manutenção.*

Preço & lado sombrio (trade-off): Como visão central permanente / infraestrutura do sistema integrada ao dashboard (necessidade básica de RAM/CPU desprezível; custos de tokens surgem apenas na execução ativa de ações de LLM dentro do módulo).

10. Matriz de referência: plugins Evey/Hermes controláveis

Nº & ID do plugin	Categoria técnica	Mecânica central & função no sistema operacional
78. evey-cost-guard	Segurança & guardiões	Monitoramento de custos em tempo real e desligamento rígido de orçamento por dia e tarefa.
79. evey-session-guard	Segurança & guardiões	Monitora durações de sessão, limites de tokens e evita sessões zumbis.

80. evey-email-guard	Segurança & guardiões	Verifica e-mails recebidos antes da triagem contra phishing e prompt-injections.
81. evey-watchdog	Segurança & guardiões	Verifica processos ativos em ritmo de 30s e reinicia automaticamente workers bloqueados.
82. evey-cache	Desempenho & cache	Cache de respostas de 24h para responder consultas repetidas de imediato e sem custo.
83. disk-cleanup	Desempenho & manutenção	Limpa artefatos temporários, logs obsoletos e libera espaço em disco.
84. evey-telemetry	Monitoramento & análises	Captura fluxos de tokens, latências e chamadas de modelos para Mission Control e Guardião.
85. evey-status	Monitoramento & análises	Ping de saúde contínuo para os 11 contêineres Docker e gateways.
86. evey-delegation-score	Monitoramento & análises	Avalia a taxa de sucesso e qualidade de passagens de tarefas entre agentes.
87. evey-autonomy	Inteligência de agentes	Controla o grau de autonomia em decisões de segundo plano de múltiplas etapas.
88. evey-council	Inteligência de agentes	Formação de maioria e consenso entre vários especialistas (acoplamento mastermind).
89. evey-reflect	Inteligência de agentes	Loop de autorreflexão para autocorreção de erros antes da entrega do resultado.
90. evey-learner	Inteligência de agentes	Extrai aprendizados duradouros de tarefas concluídas para a tabela learnings.
91. evey-identity	Inteligência de agentes	Garante o cumprimento estrito de SOUL.md, tonalidade e balizas de função.
92. evey-habits	Inteligência de agentes	Gerencia rotinas recorrentes como o briefing diário de standup das 08:00.
93. evey-proactive	Inteligência de agentes	Permite aos agentes fazer sugestões de ação por conta própria ao detectar gargalos.
94. evey-goals	Planejamento & tarefas	Vincula tarefas operacionais individuais no Kanban a objetivos empresariais superiores.
95. evey-scheduler	Planejamento & tarefas	Agendamento de tarefas em segundo plano por tempo e prioridade.
96. evey-bridge	Pontes & interfaces	Hub interno de dados entre o monólito FastAPI, o daemon Hermes e sidecars.
97. evey-mqtt	Pontes & interfaces	Protocolo leve de mensagens para sinais de status IoT e em tempo real.
98. evey-moltbook	Pontes & interfaces	Sincroniza cadernos de trabalho estruturados e notas de síntese.
99. evey-telegram-ux	Pontes & interfaces	Controla o roteamento por apelidos (/robert, /anwalt) e

		botões interativos de liberação no Telegram.
100. evey-github	Pontes & interfaces	Conexão para operações de repositório, análises de commit e rastreamento de issues.
101. evey-research	Ferramentas & pesquisa	Driver de busca profunda para web-scraping e extração de fontes no Research Lab.
102. evey-news	Ferramentas & pesquisa	Scanner em segundo plano para feeds setoriais e novidades de mercado (Oracle News Radar).
103. evey-digest	Ferramentas & pesquisa	Comprime extensas coleções de documentos em resumos gerenciais compactos.
104. evey-sandbox	Ferramentas & execução	Ambiente de execução isolado para código Python e de análise gerado no Showroom.
105. evey-rag	Memória & conhecimento	Driver de recuperação vetorial para conectar AnythingLLM e arquivos locais de conhecimento.
106. evey-memory-adaptive	Memória & conhecimento	Ponderação dinâmica de memória de curto e longo prazo conforme a pressão de contexto.
107. evey-memory-consolidate	Memória & conhecimento	Execução noturna para consolidação e deduplicação de entradas de memória relacionadas.
108. evey-wallet	Finanças & orçamentos	Gerencia créditos de API, cotas e limites de transação relacionados a custos.
109. evey-validate	Qualidade & execução	Verifica saídas quanto a conformidade de esquema, marcadores obrigatórios (SYNTHESE-BEREIT) e especificações de formato.
110. evey-verification	Qualidade & execução	Checagem de fatos e contraprova de declarações críticas antes da liberação final.
111. evey-delegate-model	Qualidade & execução	Seleciona automaticamente em subtarefas o modelo-alvo de custo-benefício ótimo.

11. Arquitetura enterprise & de resiliência

78. Replay & depuração com viagem no tempo (backend/time_travel_debugger.py)

Problema: Em workflows complexos de agentes com muitas chamadas de ferramentas, o diagnóstico posterior de falhas no log de texto é confuso e a ramificação de fluxos de execução é impossível.

Mecânica: Salva cada subetapa com entradas/saídas, snapshots e consumo de tokens em 'execution_traces'. Permite replay interativo (GET /api/traces/{task_id}), bifurcação de estado a partir da etapa N (POST /api/traces/{task_id}/fork) e rollback em 1 clique de arquivos modificados.

Benefício: Análise rápida de causas e experimentos e-se sem risco com prompts ou modelos alternativos sem reiniciar todo o pipeline.

Preço & lado sombrio (trade-off): Aumenta o I/O de disco em ~5–15 ms por edição de arquivo e ocupa, conforme o limite de snapshots, ~50–250 MB adicionais de espaço em disco.

Quando desligar? & limites: Desligar em sistemas com espaço em disco extremamente escasso ou em gerações massivas de arquivos (p. ex. artefatos de build). | Limites: máx. de snapshots: < 50 = histórico curto | > 200 = maior consumo de disco (padrão: 50). Máx. de trace-steps: padrão 100.

79. Matriz de avaliação multimodelo & comparador A/B (backend/model_benchmarker.py)

Problema: Atribuições de modelos a agentes frequentemente se baseiam em suposições subjetivas por falta de valores empíricos de comparação.

Mecânica: Envia o mesmo prompt simultaneamente a dois LLMs configurados (POST /api/router/benchmark), mede latência, consumo de tokens e comprimento da resposta e salva o histórico persistentemente em 'model_benchmarks' (GET /api/router/benchmark/history).

Benefício: Seleção objetiva de modelos por função de agente com base em dados (p. ex. DeepSeek V3 para velocidade/custo vs. Claude 3.7 para arquiteturas complexas).

Preço & lado sombrio (trade-off): N modelos candidatos custam N vezes os tokens de API por execução de teste.

Quando desligar? & limites: Quando desligar? Iniciar apenas de forma direcionada para comparações de modelos, não em operação contínua.

80. Portão de consenso tri-agente & tribunal arbitral (backend/consensus_gate.py)

Problema: Intervenções críticas (refatorações de código, migrações) trazem, em decisões individuais, o risco de alucinações ou brechas de segurança.

Mecânica: Convoca um colegiado de três membros (arquiteto de sistemas, oficial de segurança, líder de QA) (POST /api/consensus/council-vote). Cada revisor vota independentemente; a liberação exige maioria de 2/3. Todas as decisões são arquivadas em 'consensus_decisions' (GET /api/consensus/history).

Benefício: Minimização confiável de riscos e eliminação de alucinações de ponto único de falha em alterações sensíveis do sistema.

Preço & lado sombrio (trade-off): Triplica (3x) o consumo de tokens e aumenta o tempo de resposta em +6 a +20 s. Com obrigação de unanimidade, há risco de paralisação decisória (deadlock).

Quando desligar? & limites: Desligar (ou ativar apenas a partir da prioridade 4 = crítica) quando velocidade e baixos custos de API têm prioridade. | Limites: a partir da prioridade: prio 2 = quase cada tarefa custa 3x tokens | prio 4 = só tarefas críticas (padrão: prio 3). Maioria: maioria de 2/3 recomendada.

81. Autocorreção autônoma de tarefas com rollback Git (backend/task_self_healer.py)

Problema: Erros de sintaxe ou imports defeituosos de código bloqueiam tarefas de imediato e exigem intervenção manual do usuário.

Mecânica: Isola o stacktrace, realiza em erros de execução um rollback de Git dos arquivos modificados e entrega ao agente um prompt estruturado de correção (POST /api/healing/attempt). Até 3 tentativas autônomas de reparo; consulta de logs via GET /api/healing/logs.

Benefício: Resiliência máxima em execuções de código e scripts com autorreparo automático antes do escalonamento ao quadro HITL.

Preço & lado sombrio (trade-off): Cada tentativa de autocorreção custa 1 chamada LLM adicional (+3 a +12 s de latência & custos de tokens). Um rollback automático de Git descarta em caso de falha também sucessos parciais úteis.

Quando desligar? & limites: Desligar o rollback de Git quando rascunhos inacabados de código devem permanecer no diretório de trabalho para continuação manual. | Limites: máx. de tentativas de reparo: 1 = desiste de imediato | > 5 = cascata cara de reparo (padrão: 3).

82. Malha de eventos & observador de diretórios de entrada (backend/event_mesh.py)

Problema: Tarefas precisavam até agora ser iniciadas manualmente pelo dashboard ou consultadas por polling IMAP temporizado.

Mecânica: Barramento central de eventos de entrada para webhooks externos (POST /api/events/inbound/{channel}) e observador automático de diretórios (POST /api/events/scan-directory), que converte novos arquivos em 'data/inbox/' direto em tarefas de triagem Kanban.

Benefício: Captura plenamente automática de eventos e documentos de sistemas externos (GitHub, n8n, Stripe, scanners locais de arquivos) sem interação do usuário.

Preço & lado sombrio (trade-off): Intervalos de varredura muito curtos (< 3 s) geram polls contínuos do sistema de arquivos; o standup matinal diário consome ~1.500–3.000 tokens por execução.

Quando desligar? & limites: Desligar quando a pasta local de inbox não é usada ativamente ou nenhum relatório

matinal de briefing é desejado. | Limites: intervalo de varredura: 1–2s = polls desnecessários de disco | > 30s = detecção atrasada de arquivos (padrão: 5s). Horário do standup: padrão 08:00.

12. Arquitetura de autonomia, sandbox & governança

83. Sandboxes efêmeras de código (backend/ephemeral_sandbox.py)

Problema: A execução de código de agentes traz o risco de travamentos do sistema, loops infinitos ou alterações indesejadas de arquivos no host.

Mecânica: Executa código em microcontêineres Docker isolados e de curta duração com limitação rígida de recursos (512 MB de RAM, 1 CPU, timeout de 30s, sem rede) (POST /api/sandbox/execute). A sandbox é excluída sem rastros após a conclusão.

Benefício: Segurança absoluta e estabilidade do sistema na execução e teste automatizados de código.

Preço & lado sombrio (trade-off): Partida a frio de contêineres custa ~300–800 ms de latência adicional por chamada de ferramenta. Com a rede desativada, pip install ou buscas externas de API no script falham de imediato.

Quando desligar? & limites: Desligar (ou ativar a rede) quando scripts locais confiáveis precisam de acesso direto ao host ou instalações rápidas de pacotes. | Limites: limite de RAM: < 256 MB = OOM-kill com Pandas/NumPy | > 2048 MB = risco à RAM do host (padrão: 512 MB). Timeout: < 10s = cancela builds (padrão: 30s).

84. Consolidação noturna de memória (backend/memory_consolidator.py)

Problema: Traces, heartbeats e dados de diálogo obsoletos em crescimento contínuo tornam consultas ao banco de dados lentas em operação contínua.

Mecânica: Remove em fases de repouso (padrão 03:00) traces obsoletos e logs não críticos conforme prazos configuráveis de retenção (settings.json). Totalmente controlável pelas configurações Hermes Core (horário, dias de retenção, profundidade de pruning, botão manual de execução).

Benefício: Bancos de dados permanentemente enxutos, latências reduzidas e higiene automática do sistema sem esforço manual de manutenção.

Preço & lado sombrio (trade-off): Durante a execução de consolidação (~10–30 s) surge breve carga de CPU/disco; um limite de pruning baixo demais (< 0,88) pode resumir fatos relacionados, porém independentes.

Quando desligar? & limites: Desligar quando traces brutos integrais precisam ser permanentemente preservados sem filtro para uma auditoria forense. | Limites: retenção: < 7 dias = perda de trilhas importantes de auditoria | > 60 dias = BD SQLite grande (padrão: 14 dias). Limite de pruning: 0,95 = conservador | 0,88 = agressivo (padrão: 0,92).

85. Agente web visual com auditoria de screenshots (backend/visual_web_agent.py)

Problema: Tarefas em portais web complexos sem API (SPAs, formulários, dashboards interativos) não podem ser atendidas com raspadores de texto.

Mecânica: Controla um navegador headless Chromium real (POST /api/browser/browse-and-capture), captura screenshots para cada etapa de clique e entrada em .hermes/visual_traces/{task_id}/ e extrai dados estruturados.

Benefício: Rastreabilidade visual integral e interação automatizada com aplicações web modernas sem acesso a API.

Preço & lado sombrio (trade-off): O Chromium headless precisa de ~300 MB de RAM e ~3–5 s de renderização por execução de auditoria ativa.

Quando desligar? & limites: Quando desligar? Em operação puramente de backend sem verificações de UI.

86. Sala de guerra virtual & teste de estresse pré-mortem (backend/war_room_simulator.py)

Problema: Riscos imprevistos de segurança, aceitação ou conformidade em lançamentos de produtos muitas vezes só são notados após o rollout.

Mecânica: Simula três partes adversárias (grande cliente crítico, pentester ofensivo, auditor rigoroso de conformidade) antes da liberação (POST /api/simulation/war-room). Cria uma matriz agregada de riscos (LOW a CRITICAL) com exigências vinculantes.

Benefício: Descoberta proativa de fraquezas e prevenção de decisões erradas custosas antes da entrada em produção.

Preço & lado sombrio (trade-off): Custa ~1.500–3.000 tokens adicionais e +4 a +10 s de espera antes da tarefa

propriamente dita. Com limite de bloqueio MEDIUM, falsos alarmes ocorrem com mais frequência.

Quando desligar? & limites: Desligar o pré-mortem automático no dia a dia normal de desenvolvimento e usá-lo apenas para intervenções críticas de produção (HIGH/CRITICAL). | Limites: limite de bloqueio: MEDIUM = bloqueia frequentemente por precaução | CRITICAL = para apenas riscos catastróficos (padrão: HIGH).

87. Contratos de SLA & passagem entre agentes (backend/sla_contract_enforcer.py)

Problema: Delegações frouxas em hierarquias multiagentes frequentemente levam a contribuições incompletas, escalonadas ao operador humano.

Mecânica: Fecha em delegações internas contratos digitais com campos obrigatórios, comprimento mínimo e critérios de aceite (POST /api/contracts/create). Valida entregas de forma autônoma (POST /api/contracts/validate) e recusa automaticamente resultados deficientes para correção.

Benefício: Cumprimento rígido de padrões de qualidade na equipe de agentes e prevenção de etapas intermediárias inacabadas no quadro HITL.

Preço & lado sombrio (trade-off): Se o comprimento mínimo for definido alto demais (> 300 caracteres), respostas curtas precisas são rejeitadas e disparam loops caros de retrabalho.

Quando desligar? & limites: Desligar a rejeição automática quando subagentes também podem retornar respostas concisas de status ou sim/não. | Limites: compr. mín.: < 30 = deixa placeholders passarem | > 300 = rejeita respostas factuais concisas (padrão: 100 caracteres). Turnaround máx.: padrão 60 min.

88. Controle central de autonomia & limites (backend/autonomy_settings.py)

Problema: Limites fixos no código do programa impedem o ajuste rápido de regras de segurança, sandbox e resiliência em operação.

Mecânica: Agrupa todos os 10 grupos de autonomia (concorrência, eliminador de loops, autocorreção, sandbox Docker, snapshots de viagem no tempo, portão de consenso, sala de guerra, contratos SLA, inbox/standup e higiene de frotas) em settings.json e os sincroniza via GET/POST /api/settings/autonomy-modules com o System Control Center.

Benefício: Cada módulo de proteção e autonomia pode ser ligado ou desligado em tempo de execução por clique e ajustado com precisão ao perfil desejado de risco e recursos.

Preço & lado sombrio (trade-off): Um contêiner defeituoso pode entrar em crash-loop por auto-restart; a auditoria visual do navegador inicia o Chromium (+300 MB de RAM, +4 s de duração).

Quando desligar? & limites: Desligar o auto-restart durante manutenção manual de contêineres; desativar a auditoria de navegador em operação puramente de backend. | Limites: dias de backup de BD: < 3 dias = histórico escasso | > 30 dias = ~500 MB+ de armazenamento de backup (padrão: 7 dias).

13. Núcleo de performance, escala de throughput & eficiência de tokens

89. Cache semântico de prompts & respostas (backend/semantic_cache.py)

Problema: Consultas recorrentes ou levemente reformuladas de agentes causam custos desnecessários de API LLM e esperas de segundos.

Mecânica: Calcula para cada prompt um hash SHA-256 e um vetor palavra/3-gramas normalizado L2 em SQLite e entrega de imediato, com similaridade de cosseno >= 0,95, a ocorrência de resposta armazenada.

Benefício: Tempos de resposta abaixo de 10 milissegundos e 100% de economia de tokens em consultas semanticamente idênticas.

Preço & lado sombrio (trade-off): Se o limite de cosseno estiver baixo demais (< 0,92), o cache ignora diferenças sutis de significado (p. ex. 'excluir BD de teste' vs. 'excluir BD de produção') e entrega respostas erradas ou desatualizadas.

Quando desligar? & limites: Desligar em tarefas com dados em tempo real (cotações, logs ao vivo, status do sistema) ou quando cada resposta precisa ser gerada do zero. | Limites: limite de cosseno: < 0,92 = respostas erradas do cache! | > 0,98 = quase nenhuma ocorrência (padrão: 0,95). TTL: padrão 24h. Máx. de entradas: padrão 500.

90. Subtarefas paralelas map-reduce / fan-out & fan-in (backend/map_reduce_executor.py)

Problema: O processamento sequencial de muitos documentos individuais ou subperguntas leva a crescimento linear de tempo e longas esperas.

Mecânica: Divide pedidos de lista automaticamente em até N workers asyncio paralelos (controlados por semáforo) e reúne os resultados parciais na etapa de reduce.

Benefício: Multiplica a vazão em análises em lote de documentos, pesquisas e código sem sobrecarregar o event-loop.

Preço & lado sombrio (trade-off): Gera carga N vezes simultânea de API (risco de limites de taxa HTTP-429 no provedor) e ~20–35% de overhead de tokens pela síntese reduce subsequente.

Quando desligar? & limites: Desligar em limites rígidos de taxa de API ou quando as subtarefas dependem fortemente umas das outras e precisam se construir sequencialmente. | Limites: máx. de workers: > 8 = risco de limite de taxa (padrão: 4). Timeout de chunk: padrão 30s. A partir de itens: padrão 3.

91. Roteamento rápido especulativo & hedge de latência (backend/speculative_hedger.py)

Problema: Requisições lentas isoladas à nuvem (picos de latência de cauda) bloqueiam toda a execução dos agentes.

Mecânica: Inicia após o término de hedge_delay_seconds (p. ex. 2,5s) automaticamente uma chamada substituta paralela em um modelo rápido de fallback e adota o resultado que chegar primeiro.

Benefício: Elimina travamentos por endpoints de API lentos e estabiliza os tempos de resposta no percentil 99.

Preço & lado sombrio (trade-off): Se o modelo primário responder logo após o início do hedge, ambas as requisições são cobradas (até 2x de custo de tokens em prompts lentos).

Quando desligar? & limites: Desligar com orçamento apertado de API ou quando modelos lentos de raciocínio são usados, que regularmente precisam de > 10 s de reflexão. | Limites: atraso de hedge: < 1,5s = quase sempre custo duplo de API! | > 8s = quase nenhuma proteção de latência (padrão: 2,5s).

92. Compactação dinâmica de contexto & sumarização hierárquica (backend/context_compactor.py)

Problema: Sessões longas de agentes incham a janela de contexto, tornam o preenchimento do prompt mais lento e aumentam drasticamente os custos por turno.

Mecânica: Extrai a partir de um limite ajustável de tokens todos os fatos concretos (objetivos, caminhos de arquivo, decisões, erros) em um bloco compacto [HIERARCHICAL_MEMORY_STATE] e mantém apenas as rodadas mais recentes em texto integral.

Benefício: Economiza 65–80% de tokens de contexto em sessões longas sem perda de dados importantes do projeto.

Preço & lado sombrio (trade-off): A execução de compressão custa uma única vez ~1.000 tokens; linhas exatas de código ou pequenos valores de detalhe de mensagens muito antigas podem se perder no resumo.

Quando desligar? & limites: Desligar quando o agente precisa recorrer literalmente a mensagens muito antigas da mesma sessão. | Limites: limite de tokens: < 3000 = esquece detalhes cedo demais | > 24000 = altos custos de prompt (padrão: 6000). Últimos turns mantidos: padrão 4.

93. Pool de conexões SQLite WAL & batching assíncrono de escrita (backend/async_write_batcher.py)

Problema: Centenas de INSERTs SQLite individuais em execuções paralelas de agentes geram picos de I/O de disco e conflitos 'database is locked'.

Mecânica: Coleta eventos de telemetria e auditoria em um buffer circular de RAM thread-safe e os grava em lote via executemany() em modo WAL.

Benefício: Reduz acessos de escrita ao disco em cerca de 80% e mantém o loop principal de eventos livre de bloqueios.

Preço & lado sombrio (trade-off): Em caso de queda brusca de energia ou kill -9 antes do próximo intervalo de flush, as entradas em buffer na RAM dos últimos 1–5 segundos se perdem.

Quando desligar? & limites: Desligar quando cada entrada de log precisa estar síncrona no disco de imediato e sem atraso de segundos. | Limites: tamanho do lote: < 5 = muitas escritas em disco | > 100 = mais perda em falha (padrão: 25). Intervalo de flush: padrão 2,0s.

14. Arquitetura moderna de memória & RAG

94. Busca híbrida com BM25 FTS5 & reciprocal rank fusion (backend/hybrid_search_engine.py)

Problema: A busca vetorial pura encontra textos semanticamente similares, mas frequentemente falha em códigos exatos de erro, nomes de funções ou números de série.

Mecânica: Combina ranking de texto completo SQLite FTS5 (BM25) e similaridade vetorial de cosseno por reciprocal rank fusion (RRF).

Benefício: Une ocorrências exatas de palavras-chave e compreensão semântica em uma única lista de ocorrências altamente precisa.

Preço & lado sombrio (trade-off): Consulta dupla ao índice aumenta a latência de busca em ~15–40 ms; um top-K alto demais (> 10) enche o prompt do agente com milhares de tokens adicionais.

Quando desligar? & limites: Quase nunca desligar — no máximo reduzir top-K a 3–5 quando os custos de tokens por consulta devem ser minimizados. | Limites: constante RRF K: padrão 60. Peso BM25: 0,0 = só vetor | 0,5 = equilíbrio | 1,0 = só palavra-chave (padrão: 0,50). Top-K: padrão 5.

95. Grafo de conhecimento local / armazenamento de triplas (backend/knowledge_graph_store.py)

Problema: Chunks planos de texto não capturam dependências estruturadas (quem gerencia qual sistema ou qual componente usa qual serviço).

Mecânica: Extrai e salva triplas sujeito-predicado-objeto com placar de confiança em SQLite e permite travessias de grafo em múltiplas etapas (multi-hop).

Benefício: Agentes reconhecem inter-relações sistêmicas e organizacionais em vários níveis.

Preço & lado sombrio (trade-off): A extração automática de triplas custa ~500 tokens adicionais ao salvar. A partir de > 2 hops de travessia, o número de nós vizinhos explode exponencialmente.

Quando desligar? & limites: Desligar a extração automática em chat puro de curto prazo sem construção de conhecimento de longo prazo; limitar hops a no máximo 2. | Limites: máx. de hops: 1 = vizinhos diretos | 2 = ideal | >= 3 = explosão de nós! (padrão: 2). Confiança mín.: padrão 0,70.

96. RAG corretivo agêntico / CRAG (backend/corrective_rag.py)

Problema: O RAG clássico repassa ocorrências irrelevantes ao modelo de linguagem sem verificação e assim provoca respostas erradas.

Mecânica: Avalia cada chunk de documento recuperado contra um limite de relevância (0,45) e realiza em ocorrências fracas automaticamente uma reescrita de consulta com expansão de sinônimos.

Benefício: Filtra ruído de conhecimento antes de gerar respostas e aumenta a qualidade das ocorrências em perguntas vagas.

Preço & lado sombrio (trade-off): Cada reformulação de consulta custa 1 chamada LLM adicional (+1,5 a +4 s de latência). Se o limite de relevância estiver alto demais (> 0,70), o CRAG dá voltas extras desnecessárias.

Quando desligar? & limites: Desligar quando a base de conhecimento é pequena e bem mantida ou tempos rápidos de resposta abaixo de 2 segundos são exigidos. | Limites: limite de relevância: < 0,30 = deixa fontes fracas passarem | > 0,70 = loops infinitos de busca (padrão: 0,45). Máx. de reescritas: padrão 2.

97. Memória central em 2 estágios nos moldes MemGPT/Letta (backend/core_memory_manager.py)

Problema: Preferências importantes de usuários e objetivos de projeto se perdem assim que saem da janela efêmera de chat.

Mecânica: Ancoras blocos editáveis de memória de trabalho fixada (<CORE_WORKING_MEMORY>) permanentemente no prompt do sistema, separados do grande armazenamento de arquivo.

Benefício: Garante uma memória central permanente e mantida pelo próprio agente por quantas sessões forem necessárias.

Preço & lado sombrio (trade-off): Cada bloco fixado ocupa permanentemente tokens de prompt em absolutamente cada chamada LLM (10 blocos × 800 caracteres ≈ 2.000 tokens de carga básica por mensagem).

Quando desligar? & limites: Desligar ou reduzir o número de blocos a 3–5 quando modelos pequenos com janela estreita de contexto ou orçamentos apertados de tokens são usados. | Limites: máx. de blocos: > 15 = alto consumo

básico de tokens (padrão: 10). Máx. de caracteres/bloco: < 300 = corta regras | > 1500 = entope o prompt (padrão: 800).

98. Fixação automática de citações & fontes (backend/citation_pinner.py)

Problema: Sem referências diretas de fontes, o usuário não consegue verificar de qual documento provém uma declaração factual do agente.

Mecânica: Atribui frases da resposta aos documentos-fonte recuperados por sobreposição de tokens, insere evidências inline ([fonte: ID]) e verifica a cota mínima de evidências.

Benefício: Torna cada declaração rastreável à prova de auditoria e revela imediatamente alegações sem comprovação.

Preço & lado sombrio (trade-off): Aumenta o consumo de tokens de saída em ~10–15%; uma cobertura mínima alta demais (> 80%) leva a alertas falsos em perguntas criativas ou de resumo.

Quando desligar? & limites: Desligar em brainstorming livre, geração de código ou chat casual sem referência documental. | Limites: cobertura mín.: < 30% = declarações sem comprovação | > 85% = alertas falsos em resumos (padrão: 60%).

15. Execução confiável & determinismo (durable execution)

99. Saídas estruturadas & garantia de esquema JSON (backend/structured_output_enforcer.py)

Problema: Modelos de linguagem às vezes entregam JSON inválido (moldura Markdown, parênteses ausentes ou tipos errados de dados), travando pipelines posteriores de código.

Mecânica: Repara erros de sintaxe na string bruta automaticamente e impõe todos os campos obrigatórios e tipos de dados do esquema-alvo fornecido.

Benefício: Garante payloads JSON 100% legíveis por máquina e com tipos seguros entre agentes e interfaces de backend.

Preço & lado sombrio (trade-off): Cada loop de autorreparo custa +1 a +3 s de latência e tokens adicionais. A validação rígida de esquema rejeita respostas até por campos adicionais inofensivos.

Quando desligar? & limites: Desligar a validação rígida quando endpoints de API ou scripts também conseguem lidar com campos JSON flexíveis ou parciais. | Limites: verificação de esquema: rígida vs. tolerante (padrão: rígida). Máx. de reparos: 1–2 = repara 98% de todos os erros de digitação | > 3 = desperdiça tokens (padrão: 2).

100. Checkpointing de execução durável / pausa & retomada (backend/durable_checkpoint_manager.py)

Problema: Se um pedido de pesquisa ou build de 10 etapas era interrompido na etapa 8 por uma reinicialização, todos os 7 trabalhos prévios precisavam ser repetidos a alto custo.

Mecânica: Salva após cada etapa concluída o estado integral de variáveis e progresso de forma transacional em SQLite.

Benefício: Execuções longas interrompidas retomam exatamente no último checkpoint válido sem consumir trabalho ou tokens em duplicata.

Preço & lado sombrio (trade-off): Checkpointing a cada etapa individual (N = 1) gera carga perceptível de escrita SQLite. O auto-resume na inicialização pode reativar indesejadamente tarefas desatualizadas do dia anterior.

Quando desligar? & limites: Desligar a retomada automática quando tarefas interrompidas devem primeiro ser verificadas e liberadas manualmente após reinicialização. | Limites: a cada N etapas: 1 = absolutamente à prova de falhas, mais I/O de disco | 3–5 = poupa SSD (padrão: 1). Retenção: padrão 14 dias.

101. Chaves de idempotência & deduplicação de chamadas de ferramentas (backend/idempotency_guard.py)

Problema: Em retries de rede ou loops de agentes, há o risco de que o mesmo e-mail, fatura ou ação de webhook seja disparado várias vezes.

Mecânica: Atribui para cada chamada mutante de ferramenta uma chave de idempotência SHA-256 no ledger SQLite e bloqueia chamadas subsequentes idênticas dentro do TTL.

Benefício: Evita efeitos externos duplicados de forma confiável nos moldes de gateways modernos de pagamento

(princípio Stripe).

Preço & lado sombrio (trade-off): Se o usuário quiser executar intencionalmente exatamente o mesmo comando uma segunda vez dentro do prazo de bloqueio (padrão 24h), o guarda bloqueia.

Quando desligar? & limites: Reduzir horas de TTL a 1h ou desligar brevemente quando ações idênticas precisam ser executadas várias vezes seguidas em operação de teste. | Limites: TTL (horas): < 1h = proteção curta contra duplicatas | > 48h = bloqueia repetições desejadas (padrão: 24h).

102. Dry-run / modo sombra para tarefas de agentes (backend/dry_run_simulator.py)

Problema: Pedidos de agentes novos ou complexos geralmente quer-se testar primeiro sem risco em dados reais de produção antes da execução definitiva.

Mecânica: Intercepta no modo sombra todos os comandos de escrita de arquivos, banco de dados e API e registra em vez disso um plano detalhado de prévia de mutação.

Benefício: Permite exercícios simulados sem risco e auditorias de risco antes da liberação final.

Preço & lado sombrio (trade-off): ENQUANTO ATIVO, NENHUM ARQUIVO REAL É GRAVADO E NENHUMA API REAL É EXECUTADA! Se o interruptor for esquecido, o sistema parece 'morto' para fora.

Quando desligar? & limites: Manter obrigatoriamente DESLIGADO em operação produtiva normal! Ligar apenas de forma direcionada para exercícios simulados (dry-runs) e auditorias de segurança. | Limites: interruptor principal desligado por padrão (false). Ativar apenas para execuções isoladas de teste.

103. Fila de mensagens mortas (DLQ) com gerenciador de auto-replay (backend/dead_letter_queue.py)

Problema: Webhooks com falha ou jobs assíncronos em segundo plano se perdem sem rastro em falhas breves de terceiros sem reservatório de contenção.

Mecânica: Salva cada job com falha incl. payload original e motivo do erro na tabela DLQ e o repete com backoff exponencial.

Benefício: Nenhum pedido se perde; tarefas com falha podem ser repetidas a qualquer momento individualmente ou em conjunto.

Preço & lado sombrio (trade-off): Em pedidos permanentemente defeituosos (p. ex. chave de API inválida), as tentativas consomem recursos desnecessariamente; replays defasados podem disparar ações com atraso.

Quando desligar? & limites: Desligar o auto-replay quando pedidos com falha devem ser revisados exclusivamente à mão e liberados um a um. | Limites: máx. de tentativas: 1 = cancela em falha breve | > 5 = acumula jobs mortos (padrão: 3). Base de backoff: padrão 10s (10s -> 20s -> 40s).

16. Segurança, privacidade & guardrails enterprise

104. PII-Scrubber bidirecional & reidratação (backend/pii_scrubber.py)

Problema: Se e-mails, telefones ou IBANs forem transmitidos em texto claro a LLMs externos de nuvem, há risco de violações da LGPD e vazamento indesejado de dados.

Mecânica: Substitui dados pessoais sensíveis antes da chamada de API localmente por placeholders ([EMAIL_1], [IBAN_1]) e os troca novamente na resposta recebida pelos valores reais.

Benefício: Modelos de nuvem nunca veem dados pessoais reais, enquanto o usuário local ainda recebe documentos e respostas completos.

Preço & lado sombrio (trade-off): Se o LLM modificar levemente o placeholder no texto da resposta, a retrotradução (reidratação) falha. Além disso, o mascaramento por regex às vezes pode adulterar strings de código.

Quando desligar? & limites: Desligar quando apenas modelos locais (Ollama) são usados ou o LLM precisa obrigatoriamente analisar os domínios/nomes reais de e-mail. | Limites: interruptores granulares para e-mails, telefones, IBANs e reidratação automática (todos ativos por padrão).

105. Firewall de saída & allowlisting de domínios para agentes (backend/egress_firewall.py)

Problema: Via prompt-injections ocultas em páginas web, agentes poderiam ser instruídos a enviar dados internos por requisição HTTP a servidores estranhos de atacantes.

Mecânica: Verifica cada URL de destino de saída contra a allowlist de domínios configurada e bloqueia hosts não autorizados e endereços IPv4 brutos.

Benefício: Impede a exfiltração de dados em nível de rede mesmo com entradas manipuladas.

Preço & lado sombrio (trade-off): Enquanto allowlist_only estiver ativo, cada nova página web, API ou documentação não explicitamente listada na lista de domínios é rigidamente bloqueada em pesquisas web.

Quando desligar? & limites: Desligar (ou complementar domínios necessários) quando o agente de pesquisa deve pesquisar livremente na web aberta. | Limites: domínios permitidos: allowlist separada por vírgulas. Bloqueio de IP bruto protege contra exfiltrações diretas de IP.

106. RBAC granular de agentes & jaulas de escopo de arquivos (backend/agent_rbac_jail.py)

Problema: Se cada subagente possuir acesso total de escrita a todo o diretório do projeto, um agente de frontend pode sobrescrever acidentalmente arquivos do kernel de backend ou chaves .env.

Mecânica: Vincula cada função de agente a uma lista fixa de ferramentas e a uma jaula de diretório permitida (p. ex. static/ para agentes de frontend).

Benefício: Impõe o princípio de privilégios mínimos (least privilege) para cada processo no sistema operacional.

Preço & lado sombrio (trade-off): Evita que um agente administrador edite legitimamente arquivos de configuração fora da pasta do projeto ou verifique modelos .env diretamente.

Quando desligar? & limites: Relaxar brevemente apenas para tarefas dedicadas de manutenção do sistema; a proteção .env deve permanecer sempre ativa em operação normal. | Limites: interruptores de path-jail, trava .env e trava de pastas do sistema (/etc, /root, /proc).

107. Linter determinístico de saída & verificador de políticas (backend/output_policy_linter.py)

Problema: Modelos de linguagem ocasionalmente recaem, apesar do prompt do sistema, em clichês artificiais de IA ou geram snippets arriscados de shell.

Mecânica: Verifica o texto gerado em tempo real por expressões regulares, remove aberturas bajuladoras/AI-slop e neutraliza comandos destrutivos.

Benefício: Garante precisão linguística e segurança técnica sem chamada adicional de verificação LLM.

Preço & lado sombrio (trade-off): O filtro destrutivo também pode bloquear scripts inofensivos de limpeza ou migrações desejadas de banco de dados se neles ocorrerem comandos como DROP ou rm -rf.

Quando desligar? & limites: Desligar brevemente o bloqueador destrutivo quando scripts direcionados de limpeza de servidor ou migrações de reset SQL forem criados. | Limites: interruptores para remover clichês de IA, bloquear comandos destrutivos e limpeza automática.

108. Recibos de execução à prova de manipulação / recibos SHA-256 (backend/execution_receipt_ledger.py)

Problema: Para auditorias de conformidade (EU AI Act / ISO 42001), logs simples de texto não bastam, pois podem ser alterados posteriormente.

Mecânica: Encadeia cada faturamento de tarefa (modelo, tokens, custos USD, ferramentas executadas) ao hash SHA-256 da entrada anterior em uma cadeia semelhante a blockchain.

Benefício: Qualquer manipulação posterior em protocolos de custos ou decisões chama atenção imediata na verificação de integridade.

Preço & lado sombrio (trade-off): Causa overhead mínimo de computação e memória (~2 ms e ~2 KB de memória SQLite por tarefa); a cadeia hash cresce continuamente ao longo de meses.

Quando desligar? & limites: Pode permanecer permanentemente ativado em operação produtiva; desligar apenas em microssistemas extremamente limitados de memória. | Limites: encadeamento hash (SHA-256) + detalhamento de custos por execução.

17. Auto-otimização, avaliação & inteligência de enxame

109. Otimizador automático de prompts pelo princípio DSPy/TextGrad (backend/prompt_optimizer.py)

Problema: Prompts estáticos do sistema repetem os mesmos erros continuamente enquanto nenhum humano

corrigir manualmente.

Mecânica: Avalia logs reais de erro de uma função, reconhece fraquezas sistemáticas e gera regras adicionais direcionadas de guardrail para o prompt do sistema.

Benefício: Agentes aprendem continuamente com execuções falhas e blindam suas instruções por conta própria.

Preço & lado sombrio (trade-off): Se o auto-apply estiver ativo, uma regra derivada com erro pode distorcer permanentemente o comportamento do agente (desvio de prompt) e inchar o prompt do sistema.

Quando desligar? & limites: Manter o auto-apply desligado em operação produtiva para que patches sugeridos de prompts só sejam adotados após verificação humana. | Limites: a partir de erros: 1 = reage exageradamente a erros isolados | > 8 = aprende muito lentamente (padrão: 3). Máx. de regras: > 10 = incha o prompt do sistema (padrão: 5).

110. Suíte de testes de regressão gold-standard / executor de avaliação (backend/eval_regression_runner.py)

Problema: Uma mudança aparentemente pequena no prompt do sistema ou modelo pode piorar despercebidamente regras existentes de segurança ou formato.

Mecânica: Executa antes de adotar novas configurações casos de teste automatizados gold-standard e verifica taxa de aprovação e padrões proibidos.

Benefício: Evita regressões de qualidade em operação produtiva por um portão de qualidade objetivo.

Preço & lado sombrio (trade-off): Cada execução completa de avaliação dispara todos os prompts de teste registrados contra a API de LLM (custa conforme o escopo ~5.000–25.000 tokens e 15–60 s de execução).

Quando desligar? & limites: Desligar o bloqueio de deploy quando testes experimentais rápidos de prompts devem ser realizados sem espera. | Limites: taxa mín. de aprovação: < 75% = deixa pioras passarem | 100% = bloqueia já em mínimos desvios (padrão: 90%).

111. Roteador de modelos multi-armed bandit (backend/bandit_model_router.py)

Problema: Atribuições fixas de modelos não reagem quando um provedor de nuvem de repente fica mais lento ou mostra taxas elevadas de erro.

Mecânica: Pontua cada modelo continuamente por taxa real de sucesso (60%) e velocidade (40%) e seleciona dinamicamente o melhor candidato.

Benefício: Maximiza automaticamente qualidade e velocidade de resposta com base em telemetria real de execução.

Preço & lado sombrio (trade-off): Pela taxa de exploração (épsilon, p. ex. 0,15 = 15%), o roteador testa deliberadamente e repetidamente modelos alternativos que em casos isolados podem ser mais lentos.

Quando desligar? & limites: Desligar (ou definir épsilon em 0,00) quando um modelo 100% fixo e previsível deve ser imposto para cada execução. | Limites: épsilon: 0,00 = usa teimosamente o melhor modelo | > 0,30 = testa modelos mais fracos com frequência demais (padrão: 0,15). Peso sucesso/latência: padrão 0,60 / 0,40.

112. Geração dinâmica de equipes efêmeras (backend/ephemeral_team_spawner.py)

Problema: Um único agente generalista rapidamente se sobrecarrega em grandes tarefas transversais (arquitetura + UI + segurança + QA).

Mecânica: Recruta uma força-tarefa temporária de especialistas adequada ao briefing da missão e a dissolve após a missão cumprida.

Benefício: Agrupa conhecimento especializado conforme a demanda sem overhead permanente de recursos.

Preço & lado sombrio (trade-off): Uma equipe ad-hoc de 4 pessoas multiplica o consumo de tokens (3x a 5x) e precisa de overhead adicional de coordenação entre os membros.

Quando desligar? & limites: Desligar em tarefas individuais simples ou orçamento apertado de tokens; limitar o tamanho máx. da equipe no dia a dia a 3–4. | Limites: tamanho máx. da equipe: 2–4 = ideal | > 5 = alto overhead de comunicação & 5x consumo de tokens (padrão: 4).

113. Protetor de recursos ao vivo & backpressure do host (backend/resource_backpressure.py)

Problema: Contêineres paralelos, auditorias de navegador e processos locais podem sobrecarregar a RAM do host

ou a CPU em picos de carga (OOM-kill).

Mecânica: Monitora métricas de CPU, RAM e disco em tempo real e limita automaticamente tarefas de segundo plano de baixa prioridade a partir de 85% de carga.

Benefício: Protege o servidor de forma confiável contra travamentos por sobrecarga e mantém processos centrais críticos sempre responsivos.

Preço & lado sombrio (trade-off): Se os limites estiverem definidos baixos demais (CPU < 70% ou RAM < 70%), tarefas em segundo plano são constantemente freadas já em picos normais e curtos de carga.

Quando desligar? & limites: Não desligar completamente — em vez disso, em hardware forte elevar os limites de CPU/RAM a 90%. | Limites: máx. CPU/RAM: < 70% = limita em trabalho normal | > 92% = risco de OOM (padrão: 85%). Espaço livre mín. em disco: padrão 2,0 GB.

18. Pilar 3: Economia & ERP (alfa) & suíte de mandantes

Módulo / visão	Escopo funcional & características técnicas (v0.5.1)
Endereços & CRM + sincronização PocketBase	Cadastro central de contatos, devedores (10000+) e credores (70000+), painel de contato 360°, linha do tempo multicanal, calendário/reapresentações e sincronização LinkedIn & mídia via PocketBase (portas 3005/8090/8092).
Itens & serviços (ART)	Catálogo de dados mestres a partir de ART-1001 para serviços, taxas horárias, pacotes, assinaturas SaaS e produtos físicos com gestão de estoque.
Processos & contratos	Cadeia contínua de documentos proposta (ANG-YYYY-NNNN) → pedido (AB-YYYY-NNNN) incl. pipeline ponderado de negócios, previsão de receita e radar de prazos contratuais (alarmes de 30/14/7 dias).
Contabilidade & DATEV	Documentos de saída e entrada segundo § 14 UStG, QR de pagamento GiroCode, análises financeiras, cobrança automática (Dunning Engine) e exportação EXTF/CSV para DATEV.
Corporate & isolamento de mandantes	Vários empreendimentos próprios (company_id) com bancos SQLite fisicamente separados em modo WAL, faixas próprias de numeração, papéis timbrados e editor de modelos.

19. Recursos de arquitetura, segurança & deployment (v0.5.0 – v0.5.1)

Recurso / subsistema	Implementação & benefício operacional (v0.5.1)
Arquitetura modular de roteadores	Externalização em routers FastAPI dedicados sob archon/routers/ (main.py reduzido de 16.254 para 1.875 linhas, -88%) e camada central de configuração archon/core/config.py.
Endurecimento de segurança (Archon Hardened)	Encerramento de 25 achados críticos e 68 altos de auditoria; alívio do threadpool Starlette, caminhos UI-DOM blindados e proxy Docker-admin classe A para hermes_updater.
Gestão de usuários RBAC (17.10)	Funções granulares (admin, operator, viewer) via backend/auth_service.py com hashing de senhas PBKDF2-HMAC-SHA256 de 100.000 rodadas e modo de usuário único/múltiplo.
Atualização do sistema em 2 fases (17.11)	Exibição imediata da versão/HEAD local via ?local_only=true (~0,25 s) mais sincronização assíncrona com GitHub, snapshots online SQLite pré-update e rollback em 1 clique.
Proxy AnythingLLM & stack Chroma	Incorporação perfeita de AnythingLLM à biblioteca central via /anythingllm-proxy incl. fast-path /assets/* e índice vetorial local ChromaDB.
Instalador portátil em 1 clique	Instalação inicial multiplataforma via setup.bat, setup.ps1 e install.sh com docker-compose.yml, .env.example, INSTALL_AGENT.md e documentação GitHub em 9 idiomas.

Recurso / subsistema	Implementação & benefício operacional (v0.5.1)
Portão de qualidade de 4 olhos & sandbox de código	Segunda verificação automática no Kanban antes do fechamento do ticket e sandbox interativa de código no Showroom (4.4.1) para prévia ao vivo de artefatos gerados.

20. Glossário alfabético & índice remissivo

Diretório alfabético de termos de todos os módulos centrais, componentes de arquitetura e recursos (v0.5.1) com números exatos de páginas.

Recurso / termo	Descrição resumida	Página(s)
Agendador DAG	Planejamento de execução em duas etapas de subtarefas dependentes com fan-out / fan-in paralelos.	p. 13
Anti-AI-Slop & STE100	Filtro de estilo e palavras para evitar floreios desnecessários e economizar tokens de saída.	—
Archon Hardened (v0.5.1)	Release de segurança e estabilidade com estrutura modular de roteadores e achados de auditoria encerrados.	p. 3, 31
Arquitetura de roteadores (archon/routers)	Divisão modular do backend em roteadores FastAPI enxutos (-88% do tamanho do main.py).	—
Audit-Log (cadeia hash SHA-256)	Cadeia de registros à prova de auditoria de todas as ações do sistema e dos agentes.	—
Bot Screen (navegador web)	Monitor visual de execução para interações de navegador via Playwright.	—
Busca híbrida (FTS5 + RRF)	Combinação de busca textual BM25 e busca vetorial por fusão de ranking recíproco.	p. 3, 26
Cache semântico	Armazenamento temporário de 24h de prompts semelhantes para economia de tokens.	p. 3, 24
Cadeia de documentos ERP (ANG -> AB -> RE)	Documentos vinculados de vendas e contabilidade, incl. forecast de pipeline e cobrança.	—
Chaves de idempotência & dry-run	Proteção contra execução dupla de ferramentas e modo de simulação sem acesso de escrita.	—
ChromaDB	Banco vetorial local para recuperação semântica de documentos no Pilar 2.	p. 31
Compactação de contexto & LRU-K	Descarregamento de memória e compressão de contextos longos de chat ao atingir o limite de RAM.	—
Consolidação do ciclo de sono	Deduplicação noturna e compactação de longo prazo da memória dos agentes.	—
Contratos de SLA (aplicação de contratos)	Garantias formais de handover e qualidade entre agentes especializados.	p. 24
Corporate & multiempresa	Separação física de bancos de dados de várias sociedades por company_id.	—
Decision Engine (55 frameworks)	Módulo de estratégia para debates estruturados de boardroom em 9 categorias de metodologia.	—
Estimador de custos pré-voo	Cálculo prévio dos custos esperados de tokens, incl. parada por orçamento da tarefa em caso de estouro.	p. 10
Execução durável & time-travel	Execuções longas interruptíveis com checkpoints SQLite e replay de execução passo a passo.	—
Exportação DATEV & GiroCode	Interface EXTF/CSV para dados contábeis e QR de pagamento EPC em faturas.	—
Fila de cartas mortas (DLQ)	Fila de erros com controle automático de repetição para jobs falhados.	—

Recurso / termo	Descrição resumida	Página(s)
Firewall de egress	Controle de rede e domínios de saída para chamadas de ferramentas dos agentes.	—
Gate de aprovação (HITL)	Etapa humana de verificação antes de e-mails externos, lançamentos ou acessos críticos a arquivos.	—
Gate de qualidade de quatro olhos	Verificação independente de tarefas Kanban concluídas por um agente revisor.	—
Grafo de conhecimento	Armazenamento local de triplas para relações estruturadas de entidades ao lado de dados vetoriais.	p. 16, 26
Hermes Master Orchestrator	Motor central para decomposição de metas, delegação e síntese no daemon na porta 8420.	p. 11
Malha de eventos & gatilhos webhook	Processamento orientado a eventos de webhooks recebidos e alterações de pastas.	—
MCP (Model Context Protocol)	Arquitetura padronizada de drivers para conexão segura de ferramentas.	p. 6, 14
OmniRoute AI Gateway	Gateway multiprovedor incl. roteamento free-tier, latency-hedging e arena de avaliação.	—
Otimizador DSPy / TextGrad	Melhoria automática de prompts do sistema com base em métricas reais de avaliação.	—
PBKDF2 / Argon2id RBAC	Proteção de papéis e senhas no System Control Center (aba 17.10).	—
PII-Scrubber & reidratação	Anonimização bidirecional de dados pessoais sensíveis antes de chamadas externas de LLM.	—
Plugins Evey / Hermes	Extensões modulares do sistema para memória, delegação, verificação e interfaces.	—
Pool de conexões SQLite WAL	Acesso multithread seguro de escrita ao banco via write-ahead logging.	p. 25
Proxy AnythingLLM	Incorporação local de AnythingLLM + ChromaDB via /anythingllm-proxy na biblioteca central.	p. 3, 31
Pré-mortem & gate de consenso	Teste simulado de estresse de falhas e votação de múltiplos agentes antes de aprovações críticas.	—
RAG corretivo (CRAG)	Consulta de conhecimento com autoverificação e avaliação de relevância antes de gerar a resposta.	p. 26
Research Lab	Pesquisa web e de fontes em várias etapas com triangulação de evidências.	p. 3, 6, 21
Roteador de modelos Bandit	Roteador adaptativo (multi-armed bandit) para escolha dinâmica de modelos por taxa de acerto e custo.	—
Roteador JEV (intenção 0-token)	Pré-filtro determinístico sem custo de IA para comandos curtos inequívocos.	p. 10
Roteamento de custos em 3 níveis	Seleção automática de modelos em LOW / MEDIUM / HIGH por complexidade da tarefa e orçamento.	p. 8
Sandbox efêmera de código	Runtime isolada e efêmera de contêineres para código Python e scripts com segurança.	—
System Control Center (11 abas)	Console central de administração incl. usuários/RBAC e atualização do sistema em 2 fases.	p. 8, 24

Recurso / termo	Descrição resumida	Página(s)
Token Guard & loop-killer	Proteção em tempo real contra loops infinitos (verificação Jaccard) e estouros de tokens.	—
Watchdog & supervisor de deadlocks	Monitoramento de 30 segundos de tarefas travadas, contêineres e autocura automática.	—